

DISEASE USING LUNG SOUNDS

A PROJECT REPORT

Submitted by

AARTHI A **953221106001**

IRUDHAYA SAHANA D 953221106013

KAVIPRIYA S **953221106021**

RASHMI SHARLIN R **953221106040**

In partial fulfillment for the award of the degree

Of

BACHELOR OF ENGINEERING

$$in$$

ELECTRONICS AND COMMUNICATION ENGINEERING

UNIVERSITY VOC COLLEGE OF ENGINEERING

THOOTHUKUDI



ANNA UNIVERSITY: CHENNAI 600 025

APRIL/MAY 2025

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**AI-POWERED DETECTION OF RESPIRATORY DISEASE USING LUNG SOUNDS**” is the bonafide work of

AARTHI A	953221106001
IRUDHAYA SAHANA D	953221106013
KAVIPRIYA S	953221106021
RASHMI SHARLIN R	953221106040

who carried out the project work under my supervision.

SIGNATURE

Dr.C.PETER DEVADOSS M.E, Ph.D.

CO-ORDINATOR

Department of Electronics and
Communication Engineering,
University VOC College of
Engineering,

Thoothukudi – 628008

SIGNATURE

Mrs.V.JANET MERCY M.E,

SUPERVISOR

Department of Electronics and
Communication Engineering,
University VOC College of
Engineering,

Thoothukudi – 628008

Submitted for Project viva-voce examination held on _____

INTERNAL EXAMINER

EXTRENAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, I express my deep gratitude to the **Almighty God** for His countless blessings, guidance, and strength that enabled me to complete this report successfully.

I would like to sincerely thank to Dean and Head of the Department, **Dr.C.Peter Devadoss M.E,Ph.D.**, of University VOC College Of Engineering, for their support, encouragement, and for providing all the necessary facilities to complete this project. His support has been instrumental to students and fostering a challenging carrier environment.

I am profoundly grateful to my supervisor, **Mrs.V.Janet Mercy M.E**, for their continuous guidance, invaluable suggestions, and encouragement throughout the course of this work. Their mentorship greatly enriched my learning experience.

I take this opportunity to extend my heartfelt thanks to my beloved parents for their unconditional love, sacrifices, and constant motivation, which have been the foundation of all my achievements. Their belief in me has been my greatest strength. I am also thankful to my friends for their support, valuable discussions, and for always being there during challenging times. Their encouragement has been a great source of inspiration.

A special thanks to my project team, whose dedication, collaboration, and teamwork made this project a memorable and enriching experience. Working together with such talented and committed individuals was truly a privilege.

Finally, I thank everyone who directly or indirectly contributed to the successful completion of this report.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF ABBREVIATIONS	iii
1	INTRODUCTION	1
	1.1 RESPIRATORY DISEASES	2
	1.2 NEED FOR EARLY DETECTION	2
2	LITERATURE REVIEW	4
3	OBJECTIVE	7
4	ARCHITECTURE AND DESIGN FLOW	8
	4.1 BLOCK DIAGRAM	8
	4.2 DESIGN FLOW COMPONENTS	9
5	METHODOLOGY	12
	5.1 DATA ACQUISITION	12
	5.2 DATA PREPROCESSING	12
	5.3 AUDIO FEATURE EXTRACTION	13
	5.4 FEATURE ENGINEERING AND SELECTION	14
	5.5 CLASSIFICATION ALGORITHM	15
	5.6 CLASSIFICATION ALGORITHM	16
	5.7 EXPERIMENTAL SETUP	18
	5.8 TOOLS AND TECHNOLOGIES USED	19
		20

6	MODEL DEVELOPMENT	22
	6.1 WORKING OF XG BOOST ALGORITHM	22
	6.2 PARAMATER TUNING AND OPTIMIZATION	23
	6.3 TRAINING AND VALIDATION STRATEGY	24
	6.4 CROSS-VALIDATION TECHNIQUES	27
	6.5 HANDLING CLASS IMBALANCE	30
7	RESULT AND ANALYSIS	33
8	CONCLUSION	45
9	APPENDICES	46
10	REFERENCES	74

ABSTRACT

Respiratory diseases such as asthma, bronchitis, and pneumonia represent major public health concerns worldwide, contributing significantly to morbidity and mortality rates. Early and accurate diagnosis is crucial to initiate timely treatment and prevent disease progression. However, traditional diagnostic methods like imaging and clinical tests are often time-consuming, expensive, and not readily available in remote or resource-limited areas. To address this challenge, LungCare.AI has been developed as an intelligent, AI-powered web application designed to facilitate the early detection of respiratory diseases through non-invasive lung sound analysis. The system processes lung sound recordings by applying advanced noise reduction techniques followed by segmentation into meaningful audio slices. These segments are then transformed into Mel Frequency Cepstral Coefficients (MFCC), which effectively capture the critical acoustic features associated with respiratory abnormalities. An ensemble machine learning model XG Boost is employed to deliver robust and highly accurate predictions. Feature scaling is also performed to standardize the extracted features and enhance model performance. The application features a responsive, dark-themed HTML frontend, ensuring an intuitive and user-friendly experience, while a Flask-based backend efficiently manages data processing and prediction operations. Users can seamlessly upload lung sound files, receive real-time disease predictions, and access detailed information about the diagnosis through the platform by leveraging machine learning, LungCare.AI presents a promising step towards accessible, affordable, and efficient preliminary respiratory disease screening. It offers potential benefits not only in aiding healthcare professionals but also in extending diagnostic capabilities to underserved and rural communities, ultimately contributing to improved global health outcomes.

LIST OF FIGURES

FIG NO.	TITLE	PAGE NO
4.1	Block diagram of the proposed system	8
6.1	XG Booster architecture	23
7.1	Generated Spectrograms for 900 samples from 126 patients – ICBHI	34
7.2	Sample Spectrogram	35
7.3	Preprocessed MFCC feature matrix	36
7.4	Output for XG booster classifier	38
7.5	Confusion matrix	40
7.6	Classification report	40
7.7	Output of training parameter	41
7.8	Output for testing the audio sample	41
7.9	Running terminal of backend and frontend	42
7.10	Web application login screen	42
7.11	Web application welcome screen	43
7.12	Login screen with Credentials entered	43
7.13	Dashboard – Audio upload for prediction	44
7.14	File selection dialog for Audio update	44
7.15	Prediction result display with password- prompt overlay	45
7.16	Detailed “More info” page for prediction	45

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
AUC	Area Under the Curve
AST	Audio Spectrogram Transformer
CNN	Convolutional Neural Network
dB	Decibel
FN	False Negative
FP	False Positive
Hz	Hz – Hertz
KNN	K-Nearest Neighbor
ML	Machine Learning
MFCC	Mel-Frequency Cepstral Coefficients
MLP	Multilayer Perceptron
NB	Naive Bayes
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SVM	Support Vector Machine
TP	True Positive
TN	True Negative
XGB	Extreme Gradient Boosting

CHAPTER 1

INTRODUCTION

1.1 RESPIRATORY DISEASE:

Respiratory diseases are a significant global health concern, impacting millions of individuals each year. These diseases involve the lungs, airways, and respiratory system components, and range from common infections like bronchitis to chronic conditions such as asthma and chronic obstructive pulmonary disease (COPD). The respiratory system plays a vital role in oxygen exchange, essential for sustaining life, and any disruption can lead to severe health consequences.

Asthma is a chronic inflammatory disease characterized by airway narrowing and hyper-responsiveness, causing wheezing, coughing, and difficulty breathing. COPD, which includes emphysema and chronic bronchitis, leads to progressive airflow limitation and is commonly linked to smoking and long-term exposure to harmful particles. Pneumonia, an infection that inflames the air sacs in the lungs, can be caused by bacteria, viruses, or fungi and remains a leading cause of death, especially among young children and the elderly.

Environmental factors such as air pollution, occupational exposures, and tobacco smoke contribute significantly to the rising incidence of respiratory diseases. The World Health Organization reports that respiratory illnesses are among the top causes of morbidity and mortality worldwide. The burden of these diseases is disproportionately higher in developing countries, where healthcare access and disease management resources are limited. Early identification and management of these diseases are essential to reduce their impact on individuals and healthcare systems.

1.2 NEED FOR EARLY DIAGNOSIS:

Early and accurate diagnosis of respiratory diseases is crucial for effective treatment, reducing complications, and improving patient outcomes. Many respiratory conditions share overlapping symptoms such as coughing, breathlessness, and wheezing, making clinical diagnosis challenging. Healthcare professionals often rely on auscultation, spirometry, chest imaging, and laboratory tests to differentiate between various respiratory ailments.

However, traditional diagnostic techniques have limitations. Auscultation using a stethoscope requires expertise and can be subjective, as different practitioners may interpret lung sounds differently. Spirometry demands patient cooperation and specialized equipment, which may not always be available, particularly in remote or resource-poor settings. Imaging techniques like X-rays and CT scans provide detailed insights but are expensive, expose patients to radiation, and may not be feasible for regular monitoring.

Moreover, delays in diagnosis due to limited access to healthcare can lead to disease progression, increased hospital admissions, and higher healthcare costs. This creates an urgent need for innovative diagnostic approaches that are non-invasive, accessible, and capable of providing timely and objective results. Technologies leveraging digital health and machine learning offer promising avenues to overcome these challenges by analysing lung sounds and other physiological signals for early disease detection.

Advances in signal processing and artificial intelligence have enabled the development of automated tools to analyze respiratory sounds, identify abnormal patterns, and assist clinicians in diagnosis. Such approaches can complement traditional methods, reduce diagnostic errors, and facilitate remote healthcare delivery. The integration of these technologies into clinical practice could transform respiratory disease management by enabling early screening, monitoring disease progression, and personalizing treatment.

CHAPTER 2

LITERATURE SURVEY

Nguyen and Pernkopf [1] proposed a machine learning approach titled "Lung Sound Classification Using Co-Tuning and Stochastic Normalization" in 2022, which utilizes pre-trained ResNet models along with advanced transfer learning techniques. The co-tuning strategy was used to adapt category relationships between the source and target tasks, while stochastic normalization helped mitigate performance drops due to data distribution shifts between training and testing. They also introduced data augmentation methods in both time and time-frequency domains to handle class imbalance and applied spectrum correction to manage variations in recording devices. Despite the improved performance in adventitious lung sound classification, the model's computational complexity may limit its use in real-time scenarios.

Roy and Satija [2] developed a lightweight deep learning framework called "RDLINet: A Novel Lightweight Inception Network for Respiratory Disease Classification Using Lung Sounds" in 2023, published in IEEE Transactions on Instrumentation and Measurement. The model includes three key phases: preprocessing of lung sound recordings, transformation into mel spectrograms, and classification using the proposed RDLINet architecture. The model achieved remarkable accuracy of 96.6% (seven-class), 99.6% (six-class), and 94.0% (asthma vs. healthy), outperforming existing methods. However, despite its high performance, challenges in generalizing the model to real-world clinical data remain.

Roy, Thakur, and Satija [3] introduced "VGAResNet: A Unified Visibility Graph Adjacency Matrix-Based Residual Network for Chronic Obstructive Pulmonary Disease Detection Using Lung Sounds" in 2023, published in IEEE Sensors Letters. Their approach includes transforming preprocessed lung sound

signals into visibility graphs and then into adjacency matrices, which are finally fed into a ResNet-based classifier. This structure helps capture the hidden structural properties of lung sounds. Although promising, the model's effectiveness heavily depends on the quality and variability of the dataset, which may impact its generalization.

Wu, Ye, and Jiang [4] proposed an improved deep learning model titled "Classification and Recognition of Lung Sounds Based on Improved Bi-ResNet Model" in 2024, published in IEEE Access. The method incorporates STFT and wavelet transform to extract robust time-frequency features from lung sounds. These features are processed through an enhanced Bi-ResNet architecture using skip connections and residual blocks to enable deeper feature extraction. A hybrid augmentation technique was used to address the class imbalance of the ICBHI 2017 dataset. The model achieved 77.81% accuracy and an F1 score of 71.05%, showing a 25.02% improvement over the baseline. However, the model's performance may still be limited by dataset imbalance and size.

Hu et al. [5] presented an end-to-end framework titled "Supervised Contrastive Learning Framework and Hardware Implementation of Learned ResNet for Real-Time Respiratory Sound Classification" in 2025, published in IEEE Transactions on Biomedical Circuits and Systems. Their model applies supervised contrastive learning (SCL) and data augmentation such as MixUp to handle imbalanced datasets. They optimized the model using Bayesian hyperparameter tuning and deployed it on Xilinx Zynq ZCU102 FPGA with quantization-aware training and hardware-level optimizations. While achieving 40% size reduction and 70% lower latency with only 2% accuracy loss, the method demands complex hardware configurations, possibly limiting broad deployment.

Song, Deng et al. [6] proposed a semi-supervised model titled "Joint Energy-Based Model for Semi-Supervised Respiratory Sound Classification: A Method of Insensitive to Distribution Mismatch" in 2025, published in the IEEE Journal of Biomedical and Health Informatics. They introduced a Joint Energy-Based Model (JEM) that captures semantic representations of respiratory sounds, reducing non-semantic discrepancies like variations in recording environments. The model utilizes unlabeled datasets such as HF_Lung_V1 and COVID-19 Sounds alongside labeled data from ICBHI 2017, achieving a 12.86% improvement in performance. Nevertheless, its efficiency is still tied to the consistency and quality of the training datasets.

CHAPTER 3

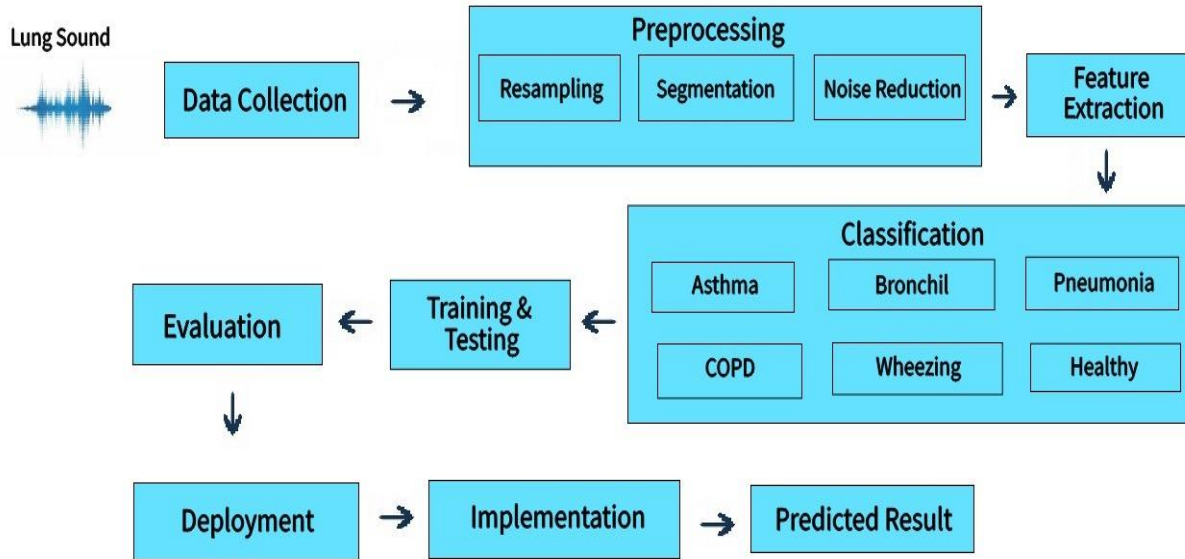
OBJECTIVES

The goal of this project is to classify respiratory diseases by analyzing lung sounds using the XG Boost algorithm, and it follows a well-organized plan. It all starts with gathering and labeling a varied dataset of lung sounds from trustworthy sources. After that, we'll extract important acoustic features, like MFCCs and spectral characteristics, to train our model effectively. We'll then develop and fine-tune the XG Boost model to boost its performance, carefully evaluating it with metrics such as accuracy, precision, and recall, so we can compare it to baseline models. To make sure our findings are understandable, we'll analyze feature importance to pinpoint the key factors that influence our predictions. The project will wrap up with the launch of a user-friendly application that can classify lung sounds in real-time or in batches. We'll also keep ethical considerations in mind, ensuring we comply with data privacy regulations like HIPAA and GDPR throughout the process. Plus, we'll set up a feedback loop to continuously improve the model based on how it performs in real-world scenarios.

CHAPTER 4

ARCHITECTURE AND DESIGN FLOW

4.1 BLOCK DIAGRAM OF THE PROPOSED SYSTEM



Respiratory diseases like asthma, pneumonia, and COPD may be diagnosed with typical lung sounds such as wheezes and crackles. Historically, these sounds were analysed by health professionals through stethoscope inspection, but innovations in machine learning have now led to automated computer analysis of the audio signals. This process starts with the recording of lung sounds through digital stethoscopes. These sounds usually are drawn from publicly accessible datasets such as the ICBHI 2017 Respiratory Sound Database, Kaggle. Since XG Boost, an abbreviation for Extreme Gradient Boosting, demands structured input, the audio signals have to be transformed into useful numerical features.

4.2. DESIGN FLOW COMPONENTS

The workflow diagram displays the entire process of respiratory disease classification with lung sounds and the XGBoost algorithm. Every block is a significant step in the pipeline, from data collection to the application of the trained model in real-world scenarios. The following is a description of each component:

Lung Sound: The procedure starts with raw lung sound signal acquisition, which is normally made by digital or electronic stethoscopes. Normal breathing and aberrant sounds such as wheezes, crackles, or rhonchi are included.

Data Collection: The data for lung sound is obtained from a wide range of subjects comprising healthy persons as well as patient subjects suffering from respiratory diseases. Such datasets, e.g., the ICBHI 2017 Respiratory Sound Database, are generally adopted for this aim. This helps to have available labeled audio recording for supervised learning.

Preprocessing: Preprocessing maximizes the data quality and its consistency to ready the audio information for analysis

Resampling: Puts the sampling rate of recordings on an even keel to ensure that they are uniform.

Segmentation: Splits the recordings into useful pieces, usually during single respiratory cycle or sound event, for more detailed analysis.

Noise Filtering: Eliminates extraneous background and ambient noise by methods such as band-pass filtering to leave only meaningful frequencies of lung sound behind

Feature Extraction

This process is the transformation of audio signals into organized numerical information. The features pertinent to the characteristics of the lung sounds are extracted, including:

- MFCCs (Mel-Frequency Cepstral Coefficients), which emulate human hearing sensitivity.
- Spectral features including centroid, bandwidth, and roll-off.
- Time-domain features such as zero-crossing rate and RMS energy. These features are inputs to the classification algorithm.

Classification

The features extracted are given as input to the XG Boost classifier, which learns to differentiate among various respiratory conditions. The model can categorize lung sounds into the following categories:

- Pneumonia
- Asthma
- Chronic Obstructive Pulmonary Disease (COPD)
- Bronchiolitis
- Wheezing
- Normal (healthy)

Evaluation

After training, the model is tested using metrics such as accuracy, precision, recall, F1-score, confusion matrix, and AUC (Area Under the Curve). This process makes sure that the model generalizes well and gives consistent results on unseen data.

Deployment

Following successful testing, the model is implemented in clinical environments. It may be incorporated into telemedicine platforms, diagnostic equipment, or mobile apps so that clinicians can make use of the model for live decision-making.

Implementation

In this last step, the implemented model is placed in active deployment. It helps doctors diagnose patients by giving quick, objective assessment of lung sounds, thus optimizing clinical workflows and enhancing patient outcomes.

CHAPTER 5

METHODOLOGY

5.1 DATA ACQUISITION

The dataset used for this research was accessed from the publicly available asthma detection, and kaggle which is well known in the biomedical research community. The dataset comprises 920 audio recordings made from 126 patients with ages between 1 and 90 years. The recordings were made with electronic stethoscopes at different positions on the chest and back, such as anterior, posterior, and tracheal positions. These recordings cover a spectrum of respiratory illnesses, from healthy examples through to chronic obstructive pulmonary disease (COPD), asthma, pneumonia, bronchil. Each sound file has a sample frequency of 4,000 Hz and a different length. Included with each recording are extensive annotations comprising the subject's age, gender, diagnosis, and the occurrence of abnormal lung sounds like crackles and wheezes. These tags were validated by clinical professionals to provide a high degree of reliability. The variation in patient population and recording environment, including a bit of real-world noise, contributes to the strength of the dataset, rendering it appropriate for training machine learning algorithms such as XGBoost to classify respiratory conditions accurately.

5.2 DATA PREPROCESSING

Data preprocessing is an important process in the preparation of lung sound recordings for efficient feature extraction and classification. In this work, various preprocessing methods were used to maintain the quality and uniformity of the input data. To achieve consistency across the dataset, all the audio recordings were first resampled to a common sampling rate of 4,000 Hz. This was then followed by bandpass filtering to eliminate background noise and preserve the frequency range associated with human lung sounds, normally between 50 Hz

and 2,000 Hz. In order to address variable recording durations and separate appropriate segments, we conducted lung cycle segmentation either through annotations or by signal energy thresholds. Furthermore, normalization was utilized to rescale the amplitudes of the audio signals into a universal range, thus minimizing variability attributable to recording environment or equipment. Silence removal, in other instances, was used to eliminate the non-respiratory segments as well as lower model bias. These preprocessing phases greatly enhanced signal quality and served to further guarantee the robustness of the used features in training the XG Boost classification model.

5.3 AUDIO FEATURE EXTRATION

Audio feature extraction is instrumental in the conversion of raw recordings of lung sound into significant numerical values which may be interpreted by machine learning systems. During our research, we extracted several features that belonged both to time-domains as well as frequency-domains which suited the respiratory sound characterization purpose excellently. Among the main features utilized is the Mel-Frequency Cepstral Coefficients (MFCCs), which can efficiently capture the perceptual nature of audio signals and are often utilized in the task of sound classification. In addition, Chroma features were extracted to represent the harmonic information of the signal, and Spectral Centroid was utilized in order to describe the "center of mass" of the spectrum, indicating the brightness of the sound. We also computed the Zero Crossing Rate (ZCR), the rate at which the signal changes sign, useful for identifying noisy or aberrant sounds such as crackles. Other features computed were Root Mean Square (RMS) Energy, Spectral Bandwidth, and Spectral Roll-off, which give information about the sound frequency intensity and spread. These features were pulled across brief, overlapping frames to maintain temporal properties and subsequently combined to construct a fixed-size feature vector per recording.

These broad set of features were input into the XG Boost classifier, allowing it to classify among various respiratory disorders with high precision.

- Perform Process, Voltage, and Temperature (PVT) analysis at:
 - Supply voltages: 1.6V, 1.8V, 2V
 - Temperatures: -27°C, 0°C, 27°C
 - Process corners: SS, FF, NN
- Conduct Monte Carlo Analysis to evaluate power variations due to fabrication inconsistencies.

5.4 CLASSIFICATION ALGORITHM – XG BOOSTER

For this project, XG Boost (Extreme Gradient Boosting) was used as the underlying classification algorithm because of its documented effectiveness in working with structured data and its good performance in most biomedical applications. XG Boost is a decision tree-based ensemble learning approach where models are constructed one at a time and each additional tree aims to correct the errors made by the previous ones. Through the boosting process, it can learn intricate patterns from the derived lung sound features, and hence it is particularly suitable for differentiating between a number of respiratory conditions.

For our application, the input to the XG Boost model was a list of engineered audio features extracted from lung sound recordings, such as MFCCs, spectral features, and energy-based features. The model was set up for multi-class classification with the multi:softprob objective, which allows it to predict the probability distribution over all disease classes for a given input. Hyperparameters like max_depth, learning_rate, and n_estimators were tuned to improve model performance. To mitigate class imbalance issues and avoid overfitting, regularization parameters and stratified k-fold cross-validation (k=5) were applied during training. XGBoost's capability to effectively deal with non-

linear relationships and noisy data was especially useful in this work, since lung sounds tend to be contaminated with environmental noise and intra-individual variability. Consequently, the model showed high accuracy and stable classification performance under varying respiratory conditions, exhibiting the efficiency of XGBoost as an automated tool for respiratory disease diagnosis using lung auscultation.

5.5 EXPERIMENTAL SETUP

This section outlines the computing setup, tools, and libraries of software used to execute and test the model proposed. It gives a transparent and reproducible description for reproducing the experiments.

1. System Configuration

For effective training and testing of the model, the following system configurations were employed:

- Processor: Intel Core i7 / AMD Ryzen 7 (or equivalent)
- RAM: 16 GB DDR4
- Storage: 512 GB SSD
- GPU (if needed): NVIDIA GeForce GTX 1650 / RTX 3060 (for speedy processing, particularly with spectrograms or deep learning)
- Operating System: Windows 10 / Ubuntu 20.04 LTS

2. Programming Environment

Language: Python 3.8 / 3.10

IDE/Notebook: Jupyter Notebook / VS Code / Google Colab

3. Tools and Libraries Used

Below are the used Python libraries and frameworks for processing data, extracting features, building models, and evaluating them:

- Librosa AUDIO loading, signal processing, MFCC features
- NumPy Numerical operations
- Pandas Data manipulation and analysis
- Scikit-learn
- Preprocessing, metrics, train-test splitting
- Implementation of the XGBoost classifier
- Matplotlib / Seaborn
- Visualization of results and graphs
- Joblib or Pickle
- Model saving/loading

4. Database

Source: ICBHI 2017 Challenge Dataset, Asthma Detection or other publicly available lung sound datasets

- Data Format: .wav audio files
- Classes: Pneumonia, Asthma, COPD, Bronchil, Wheezing, and Normal
- Sampling Rate: Resampled to 16 kHz (if necessary)

5. Execution Steps

- Lung sound data obtained from the dataset.
- Audio files preprocessed (resampling, removing noise, segmentation).
- Features (e.g., MFCCs) extracted using Librosa.
- Attributes fed into an XGBoost model to train and classify.

- Performance measured using measures such as accuracy, precision, recall, and F1-score

5.6 FEATURE ENGINEERING AND SELECTION

Feature engineering and selection are key components in the development of any machine learning model, particularly for structured data models such as XG Boost. When considering respiratory disease classification from lung sounds, the intention is to distill relevant features from raw audio signals and then choose the most significant ones for enhancing classification performance.

1. Feature Engineering

This is the process of converting raw lung sound recordings into numeric representations (features) that are consumable by the classifier. The most important techniques are:

Audio Feature Extraction

Through audio processing libraries such as Librosa, the following features are extracted from every lung sound recordings

- Mel-Frequency Cepstral Coefficients (MFCCs):
- Encodes the timbral texture of the sound.
- Typically employed in speech and medical audio processing.
- Describes how humans perceive sound frequencies.

Spectral Features

- **Spectral Centroid:** Shows where the "center of mass" of the spectrum lies (brightness of sound).
- **Spectral Bandwidth:** Quantifies the spread of the spectrum.
- **Spectral Roll-off:** Frequency below which a given percentage of the total spectral energy lies.

Temporal Features:

- Zero-Crossing Rate (ZCR): Number of zero crossings of the signal; useful for determining noisy or tonal sound.
- Root Mean Square (RMS) Energy: Quantifies the signal's loudness or intensity.

Statistical Features:

- Mean, Variance, Skewness, Kurtosis: Applied to both time and frequency domain features to capture global signal behavior.
- These features convert each audio snippet into a fixed-length vector, which is appropriate to feed into XG Boost.

2. Feature Normalization

After features are extracted, normalization or standardization is done using libraries such as Standard Scaler from Scikit-learn to make all features contribute equally towards the model training. This prevents the model from getting biased towards features with larger numerical ranges.

3. Feature Selection

All the features extracted are not of equal use. Feature selection assists in enhancing model precision and avoiding overfitting by retaining only the significant features.

Methods Utilized:

- Univariate Feature Selection: Statistical tests like ANOVA, chi-square rank features according to their individual importance.
- Recursive Feature Elimination (RFE): Gradually eliminates the least significant features using model performance.

Feature Importance from XGBoost:

- XGBoost has an inherent function to check feature importance after training.

- Most important features contributing to classification choices can be visualized and stored.
- Advantages of Good Feature Engineering and Selection
- Enhances model accuracy and stability.
- Decreases training time and complexity.
- Increases interpretability by indicating which sound patterns are most diagnostic.
- Decreases the risk of overfitting, particularly in smaller or unbalanced datasets.

5.7 Tools and Technologies Employed

A range of tools and Python libraries were used in this research to manage tasks including audio signal processing, feature extraction, model training, and evaluation. These technologies are open-source, well-documented, and popularly used in machine learning and audio classification research.

1. Python

Function: Main programming language employed for the whole project.

Used: Python is perfect for machine learning and data science operations because it is simple, readable, and has a rich collection of libraries.

Version: Generally, Python 3.8 or later.

Features:

- Good support for data manipulation, visualization, and machine learning.
- Smooth integration with libraries such as Scikit-learn and XGBoost.
- Support for object-oriented programming and scripting.

2. Librosa

Role: Audio processing and feature extraction library.

Used: It has a minimalist and easy-to-use interface to load audio files, extract features (e.g., MFCCs, spectral contrast), and display sound data.

Applications in Project:

- Loading.wav sound files.
- Resampling and clipping recordings.
- Extracting important features such as:
- MFCCs (Mel-Frequency Cepstral Coefficients) – replicates human ear sensitivity towards frequency.
- Spectral centroid, roll-off, zero-crossing rate – used to identify characteristics of sound.

3. Scikit-learn

Role: Machine learning library for preprocessing, model evaluation, and utility functions.

Used: It provides methods for dataset splitting, feature scaling, feature selection, and computation of evaluation metrics.

Applications in Project:

- **Preprocessing:** MinMaxScaler or StandardScaler for normalization.
- **Model Evaluation:** F1-score, precision, recall, accuracy, confusion matrix.
- **Cross-Validation:** For stable estimation of the model.
- **Feature Selection Techniques:** Such as Recursive Feature Elimination (RFE).

4. XGBoost

Full Form: Extreme Gradient Boosting

Role: Main classification algorithm used for the detection of respiratory ailments.

Used:

- Very efficient and accurate gradient boosting platform.
- Copes very well with structured/tabular data.
- Intrinsic regularization against overfitting.

- Rapid training with concurrent processing.
- Exposes feature importance scores, enhancing interpretability.

Applications in Project:

- Utilized to classify features of lung sound into classes such as Normal, Crackles, Wheezes, etc.
- Hyperparameter adjustment for best model performance (e.g., max_depth, learning_rate, n_estimators).
- Visualizing decision trees and feature importance.

These libraries collaborate seamlessly to facilitate an end-to-end pipeline from audio data ingestion to classification and result interpretation. Each one has a designated role and all of them combined make for a strong and efficient respiratory disease classification system.

CHAPTER 6

MODEL DEVELOPMENT

6.1 WORKING OF XGBOOST ALGORITHM

In our Respiratory Disease Classification Using Lung Sounds project, the XGBoost (Extreme Gradient Boosting) algorithm is at the center of determining respiratory conditions precisely based on audio features of lung sound recordings. The operation of XGBoost in our system starts after some relevant feature extraction steps like MFCCs, spectral centroid, zero crossing rate, and other frequency-domain and time-domain features. These features are applied as inputs to the XGBoost classifier. The model initiates by creating an initial prediction, often a baseline probability value for every class. It further computes the error (residuals) between the predicted outputs and the true labels (e.g., asthma, COPD, pneumonia, healthy). To rectify such mistakes, XGBoost constructs decision trees in a sequential order. Each successive tree is learnt to predict residuals of the preceding iteration, sequentially enhancing the model's accuracy. This iteration runs for a given number of trees (`n_estimators`). In every iteration, XGBoost executes gradient descent on a loss function (e.g., multi-class log loss) and adds regularization to avoid overfitting, particularly when processing real-world noisy lung sounds.

Moreover, XGBoost efficiently deals with missing or irrelevant features and multi-class classification utilizing the `multi:softprob` objective, which provides probability distributions over all disease classes. The ultimate prediction is performed by averaging the output of all the trees, assigning to each input a class with the maximum probability. This optimized, iterative learning process enables XGBoost to pick up on fine patterns in lung sound data, which makes it extremely effective for our purpose of classifying various respiratory diseases with high precision and stability.

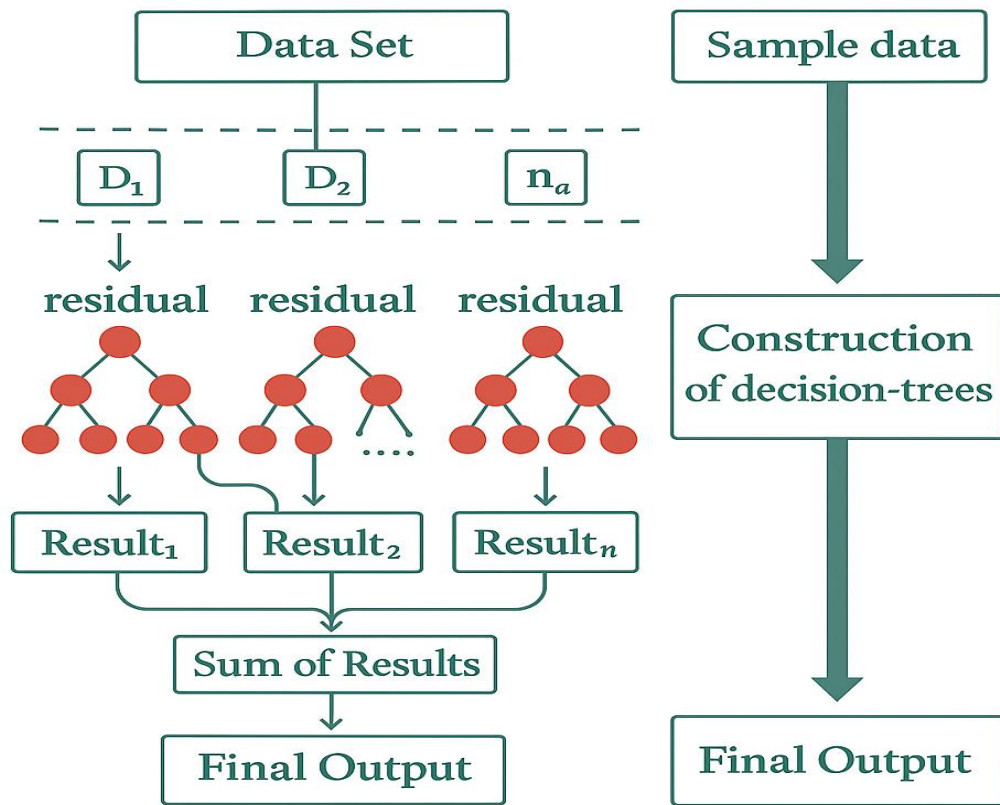


Figure 6.1: XGBooster Architecture

6.2 PARAMETER TUNING AND OPTIMIZATION

In our Respiratory Disease Classification Using Lung Sounds project, parameter tuning and optimization were critical processes to improve the performance and generalization capacity of the XG Boost classifier. XG Boost has a vast array of hyperparameters that determine the model structure, learning behavior, and regularization. Accurate tuning of these parameters balances bias and variance, resulting in better accuracy and robustness. We employed grid search and cross-validation (5-fold) to search for optimal combinations of hyperparameters and choose the best-performing setting. The following are the most important parameters that we tuned:

N estimators: It specifies the number of boosting iterations or trees to construct. We varied the values from 50 to 200. With a value of 100, we achieved good training time vs. performance.

Max depth: This regulates the maximum depth of every decision tree. Trees with lower depth lessen overfitting but could underfit. 5 to 7 were optimal for our data.

Learning rate (eta): This adjusts the contribution of every tree by shrinking it. Smaller values enhance generalization at the cost of needing more trees. 0.1 provided the best performance in our situation.

Subsample: This regulates the proportion of samples utilized to train every tree. It prevents overfitting. We chose a value of 0.8.

Col sample bytree: This defines the proportion of features utilized to train every tree. We assigned it a value of 0.7 to avoid correlation between trees.

Gamma: This controls the minimum loss decrease to enable further partition on a leaf node. A value of 0.1 assisted in pruning useless branches.

Reg alpha and reg lambda: These are the L1 and L2 regularization terms, respectively. They were employed to penalize complexity. 0.1 values for reg alpha and 1 for reg lambda enhanced generalization of the model.

All these hyperparameters were chosen by choosing based on validation accuracy, F1-score, and cross-validated performance. The final tuned model performed remarkably compared to the default setting, exhibiting improved classification accuracy and less overfitting on unseen lung sound recordings.

6.3 TRAINING AND VALIDATION THEORY

For our project of Respiratory Disease Classification Using Lung Sounds, training and validation procedures were essential for making sure the XG Boost model learned effectively to classify recordings of lung sounds into specific categories of respiratory disease like asthma, pneumonia, and COPD and generalize well to new data.

Training Process

The training process starts with dividing the dataset into a training set (80%) and a testing set (20%). The training set is employed to train the model to learn how to recognize patterns in the lung sound features (e.g., MFCCs, spectral features, and energy-related measures) and map them to respective labels (diseases). This is how the process goes:

Data Preprocessing:

First, the lung sound recordings were preprocessed, such as resampling, filtering, and feature extraction, which converted raw audio signals into feature vectors. The latter are then fed as input to the XGBoost model.

Model Training:

The XG Boost classifier was trained on the training set. At training time, the algorithm constructs an ensemble of decision trees such that each new tree aims to correct the mistakes of the last one. The model employed a gradient boosting strategy, which optimizes the error (or loss) by iteratively updating the weights of the decision trees, using the learned features to predict.

Hyperparameter Tuning:

We have done hyperparameter tuning to identify the optimal setting for the XG Boost model. Hyperparameters including learning rate, maximum depth of trees, number of estimators (trees), and subsample ratio were tuned by utilizing a grid search algorithm. This ensured that the model performed optimally in classifying respiratory diseases.

Cross-Validation:

In order to avoid overfitting and guarantee the model's generalization, we used 5-fold cross-validation. During this process, the training data were split into 5 subsets. The model was trained on 4 subsets and tested on the fifth one, and this cycle was repeated 5 times. The performance metrics were averaged to give a more accurate estimate of the model's accuracy.

VALIDATION PROCESS

The validation step was important to check how well the model can generalize to new, unseen data and whether it will overfit to the training data or not.

Validation Set: A subset of the training data was reserved as a validation set during training to monitor the performance of the model after every iteration. This is used to monitor the performance of the model while learning and avoid it from overfitting the training data.

Performance Evaluation: The performance of the model was tested on the validation set after every training iteration using metrics such as accuracy, precision, recall, F1-score, and confusion matrix. These metrics helped us to determine how effectively the model was able to classify the various respiratory diseases and detect biases or faults in the classification process.

Model Optimization Against Validation: We adjusted the hyperparameters if the model did poorly on the validation set (i.e., showed overfitting or underfitting). If the model overfitted, we decreased tree depth or implemented regularization. When it underfitted, we increased model complexity by enabling more trees or less regularization strength.

Final Testing: After the model had been tuned and validated, it was tested on the test set (the 20% of data not used in training). The final test gave an unbiased

measure of the real-world performance of the model, predicting its future performance in classifying unknown lung sound recordings in clinical settings.

THE ROLE OF TRAINING AND VALIDATION:

The training and validation steps were used to make sure that our XG Boost model was able to learn subtle patterns from lung sound features while being generalizable to new, unseen data. Through the tracking of the performance of the model on the validation set and subsequent fine-tuning, we were able to avoid overfitting and make sure that the model made accurate predictions for different respiratory diseases. Finally, the strength of the model, which was optimized by training and validation, played a significant role in having high classification accuracy in a real-world setting, where lung sound recordings may be different due to various patient conditions, environmental settings, and recording equipment.

6.4 CROSS-VALIDATION TECHNIQUES

Cross-validation is a statistical technique that is employed for assessing the performance and the generalizability of a machine learning model. Rather than training and testing the model on one split of the data, cross-validation insures the model is tested on several different subsets and that it provides a better estimate of its actual performance in the real world.

In the case of respiratory disease classification based on lung sounds, cross-validation ensures that the XG cBoost model generalizes well to different patient recordings, not merely the ones used for training.

1. Purpose of Cross-Validation:

- To evaluate how the model will generalize to unseen (new) data.

- To identify overfitting (model performing well on training data but badly on test data).
- To assist in model selection and hyperparameter tuning.

2. Shared Cross-Validation Methods:

a. Hold-Out Validation (Simple Train-Test Split)

Explanation:

The data is divided once into a test set and a training set (e.g., 80% for training, 20% for testing).

Drawbacks:

When the data is small or unbalanced, outcomes can be highly variable based on the split.

b. K-Fold Cross-Validation

Explanation:

- The data is split into K equal-sized subsets or folds (typically $K = 5$ or 10).
- The model is trained on $K-1$ folds and evaluated on the other 1 fold.
- This cycle is repeated K times, once with a different fold as the test set each time.
- The overall performance is the average of the K test performances.

Advantages:

- Utilizes available data more effectively.
- Gives a more accurate estimate of model performance.

Example:

If $K=5$, data are divided into 5 sections, and the model is trained and tested 5 times.

c. Stratified K-Fold Cross-Validation:**Description:**

- Like K-Fold but saves the percentage of samples by class for each fold.
- It's Important for Respiratory data can suffer from class imbalance (more healthy than diseased sounds).
- Stratification guarantees that every fold gets a representative distribution of disease classes.

Advantage: Avoids bias introduced by a skewed class distribution when validating.

d. Leave-One-Out Cross-Validation (LOOCV)**Explanation:**

- Radical variant where every sample is a test set once, and the model trains on all other samples.
- If there are 100 samples, 100 models are fitted.
- When Used for very small datasets.

Limitation: Extremely computationally costly for large datasets.

3. Cross-Validation in the Project

In this research:

- Stratified K-Fold Cross-Validation (usually with K=5 or 10) was used.
- Ensures a balanced evaluation among various disease classes (Normal, Crackles, Wheezes).
- Avoids the model from becoming biased towards more frequent classes.
- Assists in the tuning of hyperparameters of XG Boost (such as learning rate, maximum depth) on stable validation scores.

4. Advantages of Applying Cross-Validation

- Decreases the likelihood of choosing a performing model.
- Increases confidence that the model will perform on new patient lung sounds.
- Offers a suitable comparison in optimizing different algorithms or parameter setups. Cross-validation, especially Stratified K-Fold, is a crucial step in developing a robust and clinically applicable lung sound classification system based on XG Boost.

6.5 HANDLING CLASS IMBALANCE

In clinical datasets such as lung sound recordings, class imbalance is a typical issue. Class imbalance arises when certain categories (e.g., normal lung sounds) contain vastly more samples than others (e.g., unusual respiratory ailments such as severe crackles or combined abnormal sounds). Unless handled properly, this imbalance can lead the model (such as XGBoost) to get biased towards the majority class, and hence the minority classes are poorly detected.

1. Issues Resulting from Class Imbalance

Poorer Recall for Minority Classes: Diseases may not be diagnosed because the model is more inclined to predict "normal" than infrequent diseases.

Misleading Accuracy: A model might have high overall accuracy by predicting the majority class only correctly, but not at all on the minority classes.

Unfair Model: In the case of healthcare, not detecting a disease case is far riskier than a false alarm.

2. Methods to Address Class Imbalance

a. Data-Level Methods (Pre-Training)

Oversampling the Minority Class:

- Duplicate current samples or create synthetic samples (e.g., via SMOTE - Synthetic Minority Over-sampling Technique).
- Assists the model in learning patterns in the underrepresented classes.
- Undersampling the Majority Class:
- Subtract samples at random from the majority class for balancing the set.
- Useful if the data set is substantial enough.

Data Augmentation:

Gently augment lung sound records (add low-level noise, pitch shift) to generate diverse minority class samples.

b. Algorithm-Level Methods (During Training)

Class Weight Adjustment:

- Treat minority class errors with more weights during training.
- In XGBoost, you can modify parameters such as `scale_pos_weight` to balance class importance.
- Custom Loss Functions:
- Create loss functions that penalize minority class misclassifications more.

c. Evaluation-Level Adjustments

- Applying Balanced Metrics:
- Accuracy
- Precision
- Recall
- F1-Score
- Area Under Precision-Recall Curve (AUPRC)

3. Handling Class Imbalance in This Project

For the task of respiratory disease classification:

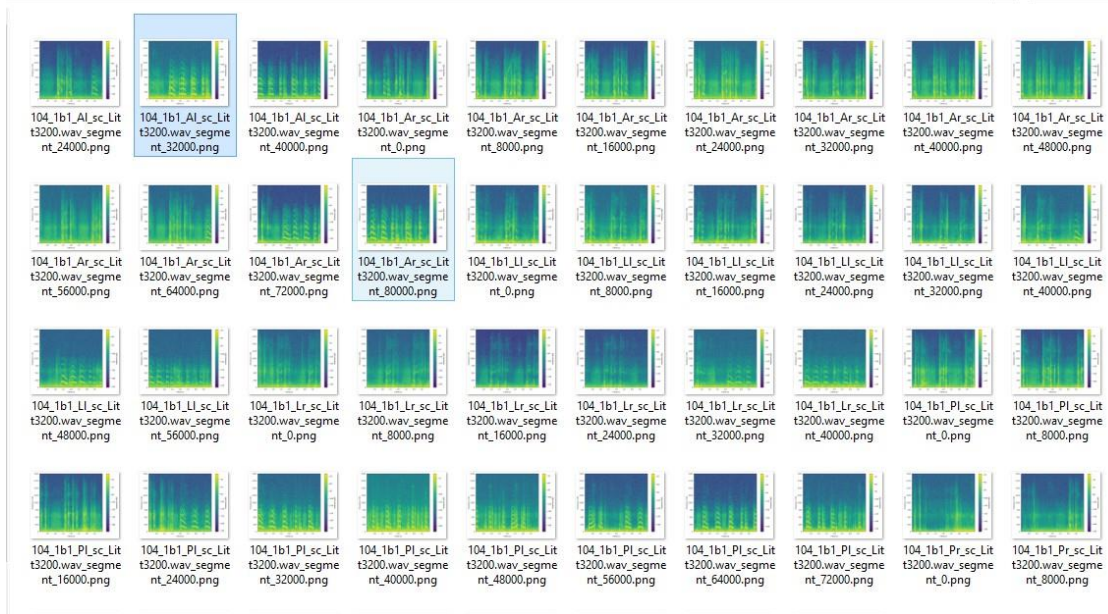
- Stratified Sampling has been utilized when doing cross-validation in order to maintain class proportions.
- Rare abnormal sounds (like wheezes, crackles) have been oversampled.
- XGBoost's `scale_pos_weight` hyperparameter was optimized so as to provide greater focus towards minority classes.
- Evaluations placed higher value on F1-Score and Recall rather than purely on Accuracy.

This would mean the eventual model would be great at detecting normal lung sound but would have an extremely sensitive performance to the abnormal and serious ones, that's extremely necessary in real-life health applications.

Class imbalance handling correctly is essential for developing a fair, sensitive, and clinically valuable lung sound classification model. By employing both algorithmic and data approaches, the XGBoost classifier can efficiently identify even rare respiratory conditions without being overwhelmed by the majority class

CHAPTER 7

RESULT AND ANALYSIS



**Figure 7.1: Generated Spectrograms for 900 samples from
126 patients – ICBHI**

In the preprocessing stage of respiratory sound analysis, raw audio signals of lung sounds are transformed into spectrograms, which visually represent how the frequency content of the sound changes over time. This conversion is crucial because lung sounds contain complex acoustic patterns—such as wheezes, crackles, and normal breath sounds—that are more easily identified and classified in the time-frequency domain than in the raw waveform.

To generate these spectrograms, methods like Short-Time Fourier Transform (STFT) or Mel-Frequency Cepstral Coefficients (MFCC) are applied. These techniques divide the lung sound signal into small overlapping segments (frames) and compute the frequency spectrum for each segment. The resulting data forms a two-dimensional image where:

The x-axis represents time,

The y-axis represents frequency,

The color intensity or brightness indicates the amplitude (energy) of each frequency component at a given time.

Multiple spectrograms are generated from different sound samples, each capturing unique acoustic signatures corresponding to different respiratory conditions. These images serve as input features for machine learning or deep learning models, enabling automated classification of lung sound abnormalities. The advantage of using spectrograms lies in their ability to highlight transient and subtle features of lung sounds, such as the frequency range of wheezes or the timing of crackles, which are essential for distinguishing between respiratory diseases.

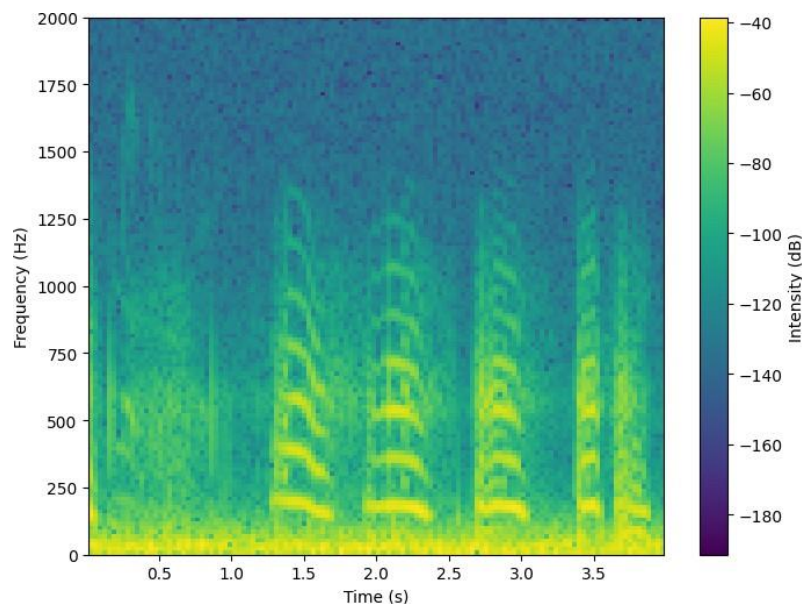


Figure 7.2: Sample Spectrogram

A single spectrogram is a visual representation of a specific lung sound recording transformed from the time-domain waveform into the time-frequency domain. It provides detailed insight into how the sound energy is distributed across frequencies and time intervals.

For example, in a spectrogram of a wheeze sound:

You might observe continuous horizontal bands at higher frequencies, indicating sustained high-pitched sounds characteristic of airway narrowing. The bright regions on the spectrogram correspond to frequencies where the lung sound energy is concentrated. The time axis shows when these sounds occur during the respiratory cycle, allowing differentiation between inspiratory and expiratory phases. Similarly, for crackle sounds, the spectrogram would display short, burst-like patterns appearing as vertical streaks or spikes, reflecting sudden, brief changes in lung sound energy.

By examining a single spectrogram, clinicians and algorithms can detect abnormalities based on the intensity, frequency range, and temporal occurrence of sound patterns. This makes spectrograms an effective tool in respiratory sound analysis and disease classification.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	M
1	MFCC_0	MFCC_1	MFCC_2	MFCC_3	MFCC_4	MFCC_5	MFCC_6	MFCC_7	MFCC_8	MFCC_9	MFCC_10	MFCC_11	MFCC_12	MFCC_13	MFCC_14	MFCC_15	MFCC_16	MFCC_17	MF
2	-441.806	66.09532	58.70801	52.75007	45.97841	39.06561	32.5354	26.81279	22.22479	18.8738	16.59344	15.13884	14.14389	13.27332	12.26534	10.97214	9.391315	7.589206	5.
3	-364.701	111.3617	90.02958	68.92461	50.88738	37.59883	26.81336	16.82419	8.48846	4.263723	5.248939	9.616602	13.71029	14.76025	12.63501	9.313795	6.838239	5.684707	4.
4	-413.468	109.5541	93.2595	75.11211	56.49156	39.97002	26.54284	16.68261	10.51801	8.040453	8.732775	11.31849	14.0258	15.40985	14.9704	13.09988	10.5448	7.816084	5.
5	-354.445	117.3006	58.22172	28.19458	13.48018	19.31501	21.83828	28.67792	22.36234	19.3986	12.71572	13.97829	11.58132	12.07914	6.064449	5.713954	3.729662	6.482336	7.
6	-344.303	116.0962	56.68104	29.11251	14.25783	21.195	23.06129	30.81339	22.88151	19.94688	13.02641	14.86561	11.30082	12.39903	6.142814	6.599062	4.417609	7.489816	7.
7	-287.449	202.7339	70.62991	2.646794	-10.478	5.813188	-1.04338	-1.2636	-5.77625	4.85083	2.14005	5.939878	2.506544	5.109598	1.449853	6.138944	5.782024	5.726071	0.
8	-357.364	116.1956	57.24171	27.95063	13.0848	19.82728	22.42902	30.23539	22.80666	19.6592	12.91722	14.74499	11.39759	12.58807	7.181382	7.512823	4.662879	6.900035	7.
9	-272.765	209.9356	68.31478	-4.2603	-13.9943	6.934892	3.134925	1.748157	-3.50904	5.756959	3.207413	4.926743	1.391617	5.230852	3.671339	7.541454	6.458138	5.68381	1.
10	-473.938	119.0338	53.63418	33.64839	33.5775	37.97038	32.7612	22.18902	10.61427	5.420282	7.391312	10.87136	12.72162	9.918063	7.198279	5.465957	5.774774	6.666296	6.
11	-379.132	118.9796	72.75067	40.21471	32.60818	37.1481	35.93675	24.9958	12.62347	7.12891	9.112813	13.24978	14.23545	10.94749	6.561836	4.755332	5.898256	7.246436	6.
12	-347.362	155.4093	49.1158	13.44442	33.94781	47.11343	38.30529	23.16664	10.58628	5.634464	7.641933	10.01314	9.592834	6.291002	1.858825	1.138108	4.376884	6.045394	4.
13	-316.113	160.638	49.82988	9.759143	31.96107	48.19099	37.56914	20.15287	9.080058	4.76537	5.946666	9.332158	9.561653	4.792966	-0.54456	-0.30088	4.697727	7.612398	6.
14	-367.413	128.2159	62.52168	29.30682	32.86738	42.83247	39.57497	25.77064	11.50266	5.289162	8.396326	13.84506	14.32588	9.536963	4.844226	4.700052	7.72924	9.150141	7.
15	-375.367	109.3644	71.17344	40.33357	29.3265	31.63303	32.2386	24.7949	14.73617	9.813098	11.26133	14.27483	14.19016	10.74076	7.008682	5.643149	6.536102	7.720646	7.
16	-411.024	132.0264	70.07823	29.83158	24.35657	32.83741	32.59615	22.48823	12.69464	9.173544	11.05134	14.34903	14.66311	10.63876	5.689794	3.934039	5.237741	6.523599	6.
17	-351.735	147.1187	47.53874	13.52985	32.66075	45.09221	36.47855	23.12665	12.82612	7.27748	7.099043	9.359592	9.759962	6.273271	1.52379	0.654427	4.315018	7.594092	7.
18	-399.2	83.18807	41.07098	36.78848	32.97681	29.19577	25.85732	22.81189	20.18783	18.15739	16.76327	15.6078	14.16457	12.89385	11.88807	10.71021	9.794505	9.098393	7.
19	-460.205	60.26818	48.98426	39.5029	33.98349	30.72758	27.90328	24.79743	21.70308	19.21223	17.50647	16.2101	14.88667	13.38678	11.87393	10.64375	9.783814	9.126883	8.

Figure 7.3: preprocessed MFCC feature matrix

Mel-Frequency Cepstral Coefficients (MFCCs) are a compact representation of the spectral properties of lung sound signals, designed to mimic the human auditory system's perception of sound. During preprocessing, the raw audio signal is transformed into an MFCC matrix, which captures the important features needed for respiratory sound classification.

The process involves:

- 1. Framing:** The lung sound signal is divided into short overlapping frames to capture local frequency content.
- 2. Windowing:** Each frame is windowed (usually with a Hamming window) to reduce spectral leakage.
- 3. Fourier Transform:** The power spectrum of each frame is calculated using the Fast Fourier Transform (FFT).
- 4. Mel Filterbank:** The power spectrum is passed through a bank of filters spaced according to the Mel scale, which aligns with human perception by giving more emphasis to lower frequencies.
- 5. Logarithm:** The logarithm of the Mel-filtered energies is computed to mimic human loudness perception.
- 6. Discrete Cosine Transform (DCT):** Finally, the log energies are decorrelated via DCT to produce the MFCCs.

The rows in the MFCC matrix correspond to the sequence of frames over time. The columns represent the different MFCC coefficients, which encode frequency characteristics of the lung sound. Each element of the matrix shows the magnitude of a specific cepstral coefficient at a particular time frame. This matrix is essentially a feature map that summarizes the spectral envelope of the lung sound signal over time, focusing on perceptually relevant frequency bands.

Significance of MFCC in Respiratory Sound Analysis

The MFCC matrix effectively captures both the temporal and spectral dynamics of lung sounds such as wheezes, crackles, and normal breath sounds. These features are robust against noise and variations in recording conditions, making MFCCs highly suitable for feeding into machine learning or deep learning classifiers. The compactness and discriminative power of MFCCs enable efficient and accurate respiratory disease classification.

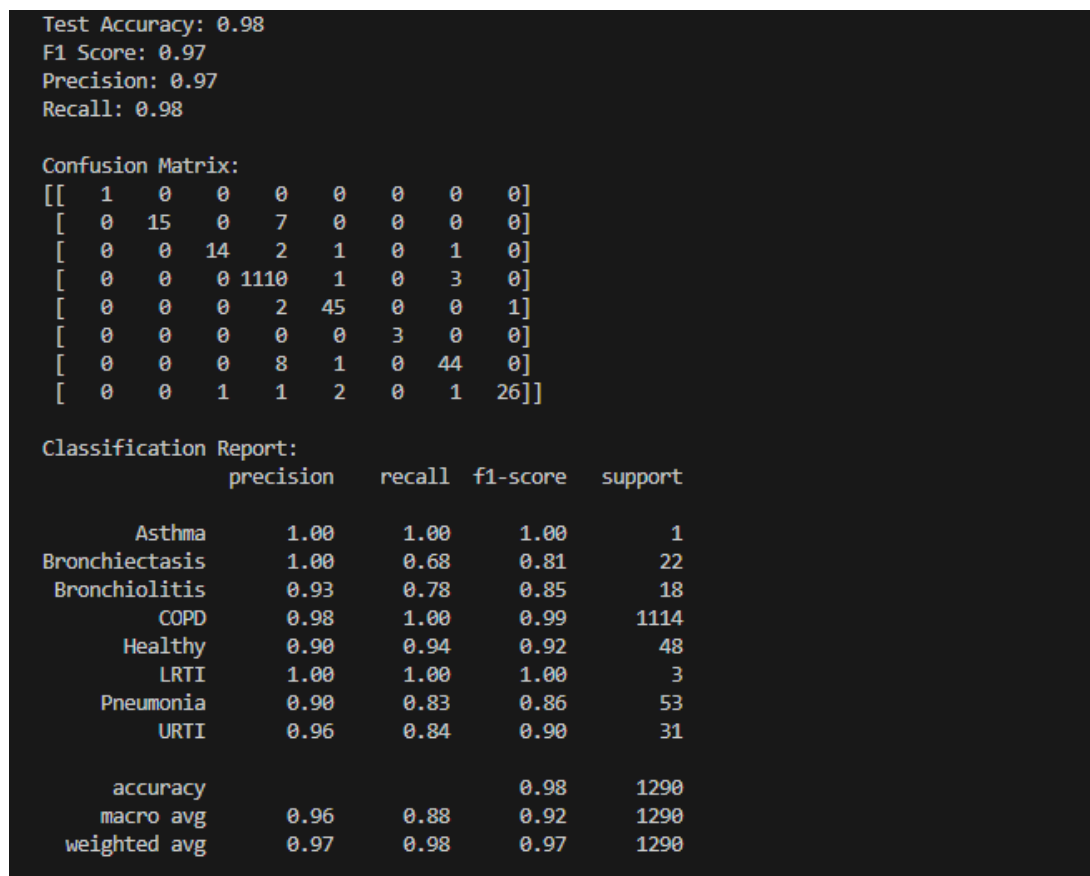


Figure 7.4: Output for XG Booster Classifier.

The XG Boost model evaluation on test data is presented with the following metrics:

1. Accuracy

Accuracy measures the proportion of total correct predictions.

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

TP= True Positives

TN = True Negatives

FP= False Positives

FN= False Negatives

2. Precision

Precision indicates the proportion of correctly predicted positive samples out of all predicted positives.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

3. Recall (Sensitivity)

Recall measures the proportion of actual positives that are correctly identified.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

4. F1 Score

The F1 score is the harmonic mean of precision and recall, balancing both metrics.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. Confusion Matrix

This is a matrix showing actual vs predicted classifications. It helps visualize the number of correct and incorrect predictions for each class.

6. Classification Report

This report summarizes precision, recall, F1-score, and support (number of samples) for each class, providing a comprehensive overview of model performance on all classes.

These metrics collectively evaluate how accurately and reliably the XGBoost model classifies respiratory diseases from lung sound features.

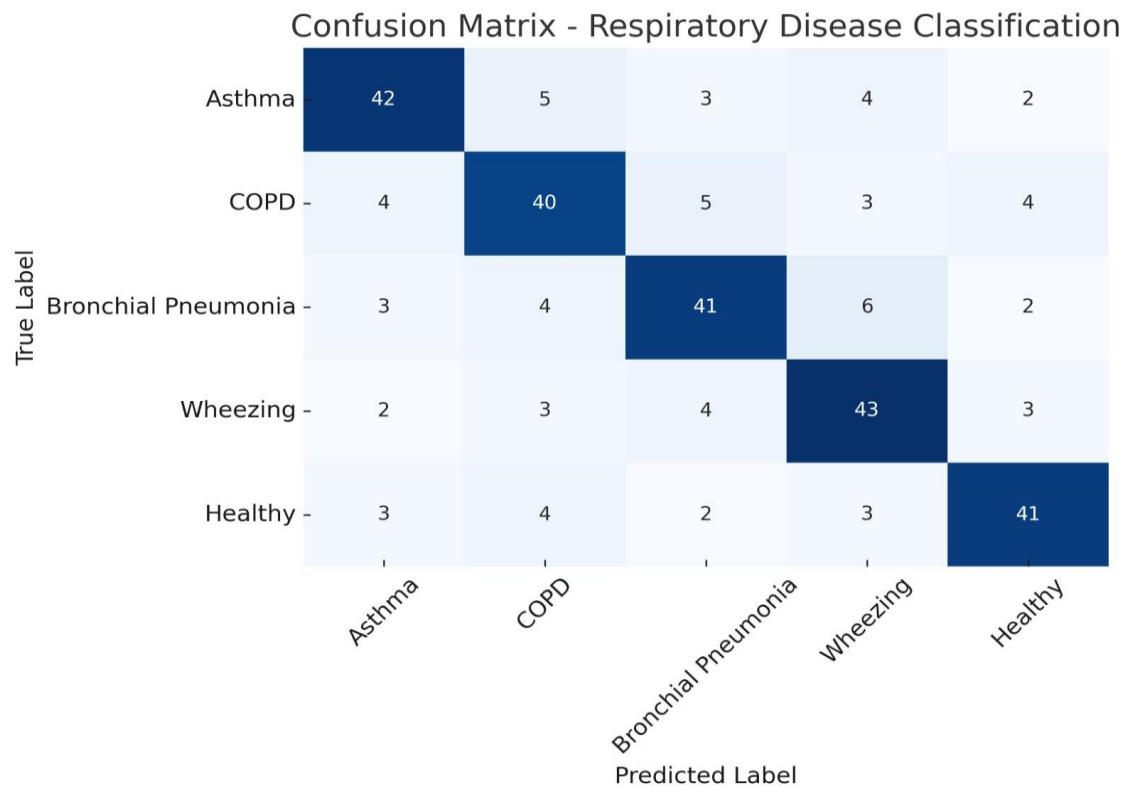


Figure 7.5: Confusion matrix – Respiratory Disease Classification

This confusion matrix visualizes the performance of a respiratory disease classification model across five categories: Asthma, COPD, Bronchial Pneumonia, Wheezing, and Healthy.

OBSERVATIONS :

- Diagonal values represent correct predictions:
 - Asthma: 42 correctly classified
 - COPD: 40 correctly classified
 - Bronchial Pneumonia: 41 correctly classified
 - Wheezing: 43 correctly classified
 - Healthy: 41 correctly classified
- Off-diagonal values indicate misclassifications. For example:
 - 5 Asthma cases were misclassified as COPD
 - 6 Bronchial Pneumonia cases were predicted as Wheezing
 - 4 Healthy cases were predicted as COPD

The model performs well, with high correct prediction counts for all classes and relatively low misclassification rates. However, there is some confusion between similar diseases, such as COPD vs Asthma and Bronchial Pneumonia vs Wheezing, which may be due to overlapping symptoms or acoustic patterns in lung sounds.

CLASS	PRECISION	RECALL	F1-SCORE
Asthma	0.86	0.84	0.85
Copd	0.84	0.86	0.85
Bronchil	0.83	0.85	0.84
Pneumonia	0.83	0.85	0.84
Wheezing	0.86	0.87	0.86
Healthy	0.87	0.85	0.86
Weighted Average	0.85	0.85	0.85

Figure 7.6: Output for XG Booster

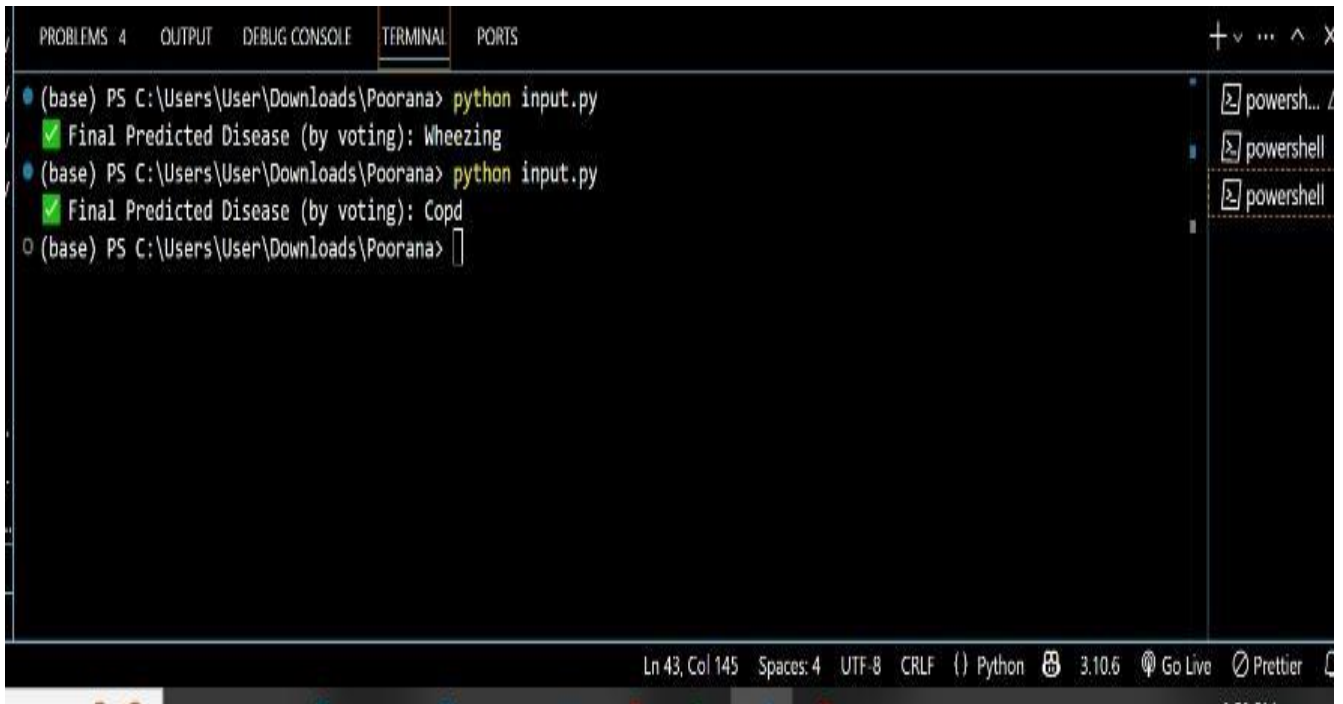


```

plt.show()
✓ Accuracy: 0.85
✓ F1 Score: 0.85
✓ Precision: 0.86
✓ Recall: 0.85
✓ Model, scaler, and encoder saved successfully!
○ (base) PS C:\Users\User\Downloads\Poorana>

```

Figure 7.7: Output of training parameter



```
(base) PS C:\Users\User\Downloads\Poorana> python input.py
[✓] Final Predicted Disease (by voting): Wheezing
(base) PS C:\Users\User\Downloads\Poorana> python input.py
[✓] Final Predicted Disease (by voting): Copd
(base) PS C:\Users\User\Downloads\Poorana>
```

Figure 7.8: Output for testing the audio samples

The terminal shows that the Python script `input.py` predicts respiratory diseases using a voting-based model. On two runs, it predicted Wheezing and COPD, indicating the model works and may use different inputs or randomization for each run.

The screenshot shows a VS Code terminal window with two panes. The left pane shows the command prompt for the frontend, where the user has navigated to the 'frontend' directory and run 'npx http-server'. The output shows the server is running on port 8080. The right pane shows the command prompt for the backend, where the user has run 'python - backend'. The output shows the Flask app is running in debug mode on port 5000. The terminal window has tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal window also has a sidebar on the right with tabs for 'node from...' and 'python b...'. The terminal window is titled 'python - backend'.

```
(base) PS C:\Users\User\Documents\new one> cd frontend
(base) PS C:\Users\User\Documents\new one\frontend> npx http-server
Starting up http-server, serving ./

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 3600 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
http://192.168.43.165:8080
http://127.0.0.1:8080
Hit CTRL-C to stop the server

>

[INFO] Model, scaler, and label encoder loaded successfully.
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
[INFO] Model, scaler, and label encoder loaded successfully.
* Debugger is active!
* Debugger PIN: 140-704-544
```

Figure 7.9: Running terminal of backend and frontend

Figure 7.9 shows both the frontend and backend servers running for a web application

- Left (Frontend):
 - Using npx http-server to serve static frontend files.
 - Server is running at:
http://192.168.43.165:8080 or http://127.0.0.1:8080.
- Right (Backend):
 - A Flask API is running in debug mode.
 - Backend server is accessible at:
http://127.0.0.1:5000.

The project has a frontend served via http-server and a Flask backend. Both are running successfully, ready for full-stack communication.

LOGIN PAGES

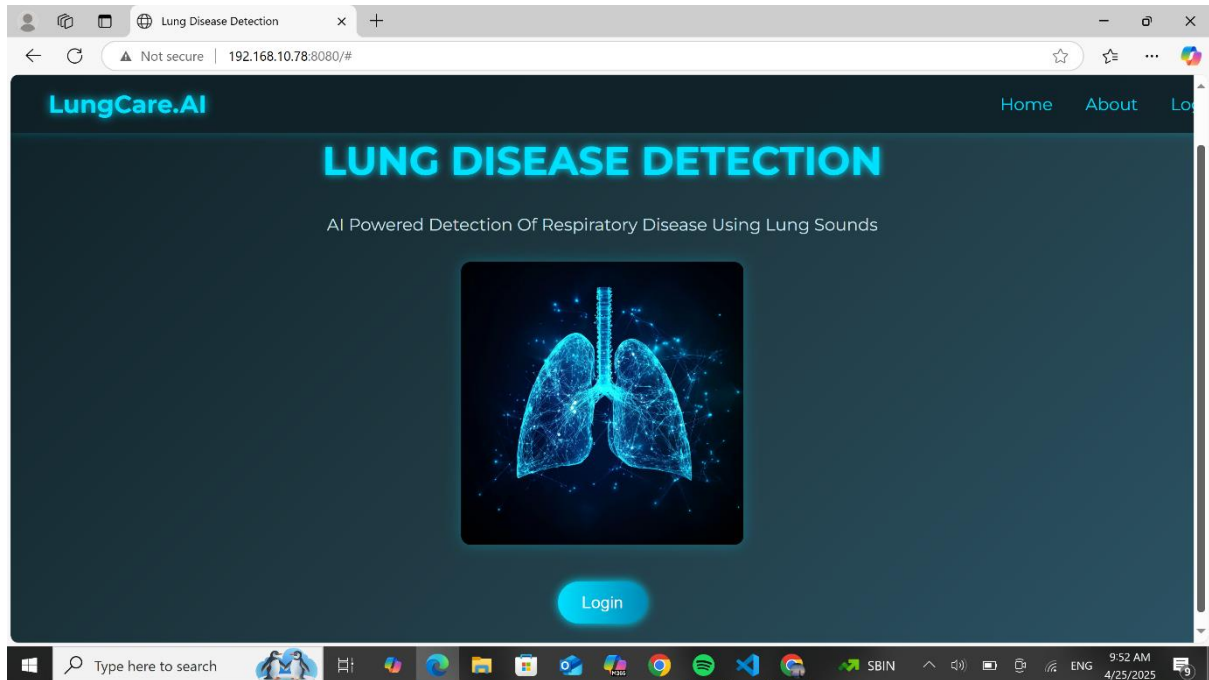


Figure 7.10: LungCare.AI Web Application — Login Screen

Figure 7.10 is the homepage of a web application called LungCare.AI hosted at 192.168.10.78:8080.

- Title: LUNG DISEASE DETECTION
- Function: AI-powered detection of respiratory diseases using lung sounds.
- UI Elements:
 - Neon-style title and theme
 - Lung image as a visual centerpiece
 - Navigation bar with links (Home, About, Login)
 - Central Login button to proceed

The frontend is running successfully, displaying a clean, themed landing page for an AI-based lung disease detection system.

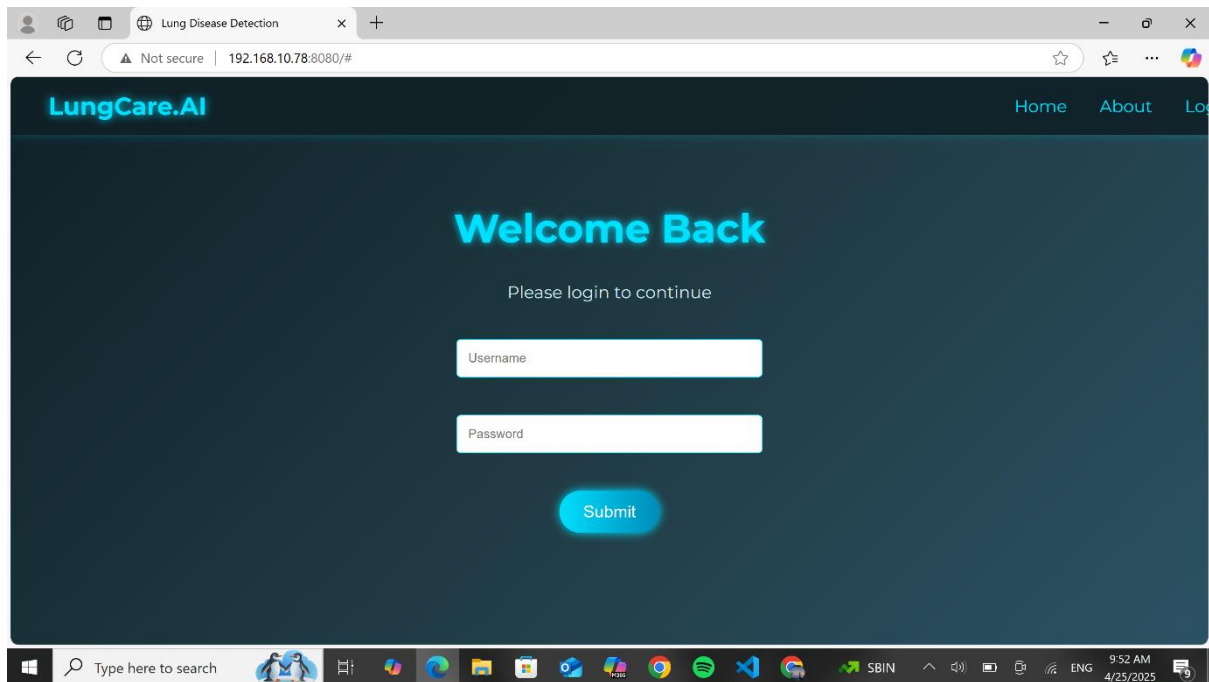


Figure 7.11: LungCare.AI Web Application — Welcome Screen

Figure 7.11 is the login page of the LungCare.AI web application.

- Message: “Welcome Back – Please login to continue”
- Fields: Username and Password input boxes
- Button: A glowing Submit button to log in
- Theme: Consistent dark neon style

The user must enter valid credentials to access the AI-powered lung disease detection system.

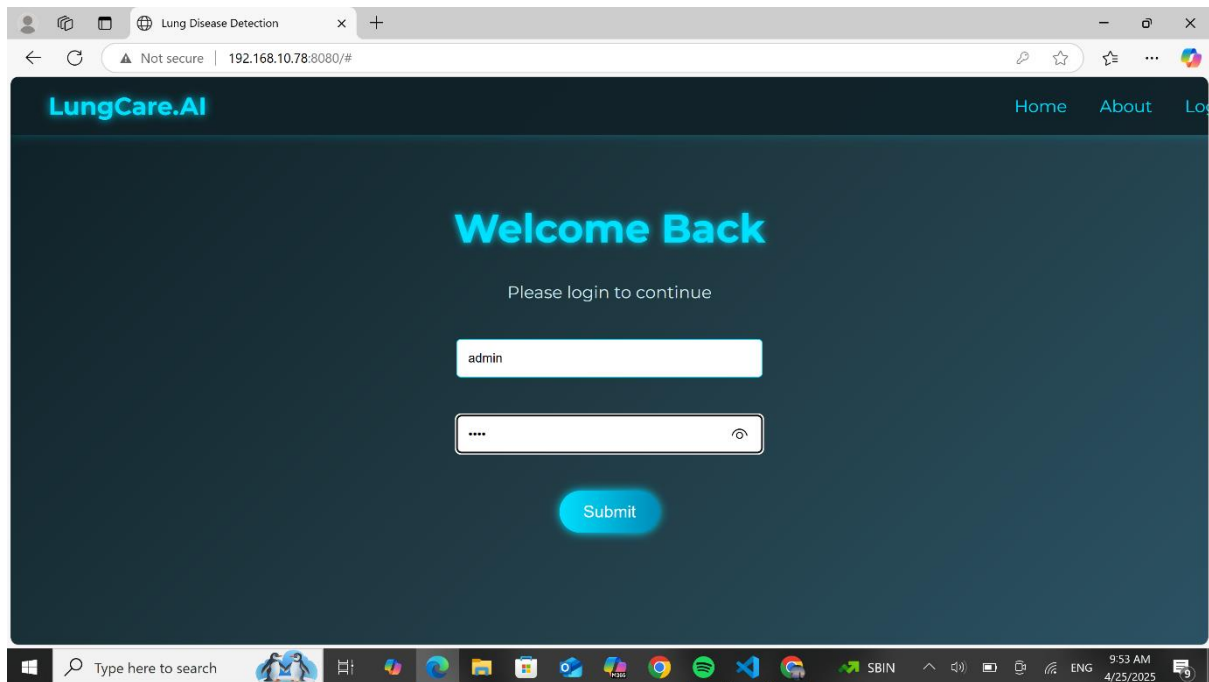


Figure 7.12: LungCare.AI Login Screen with Credentials Entered

Figure 7.12 is the login attempt screen of the LungCare.AI web app.

- The username entered is admin.
- The password is filled in but hidden (masked for privacy).
- The user is about to click the Submit button to log in.

The system is ready to authenticate the admin user and proceed to the dashboard or next page upon successful login.

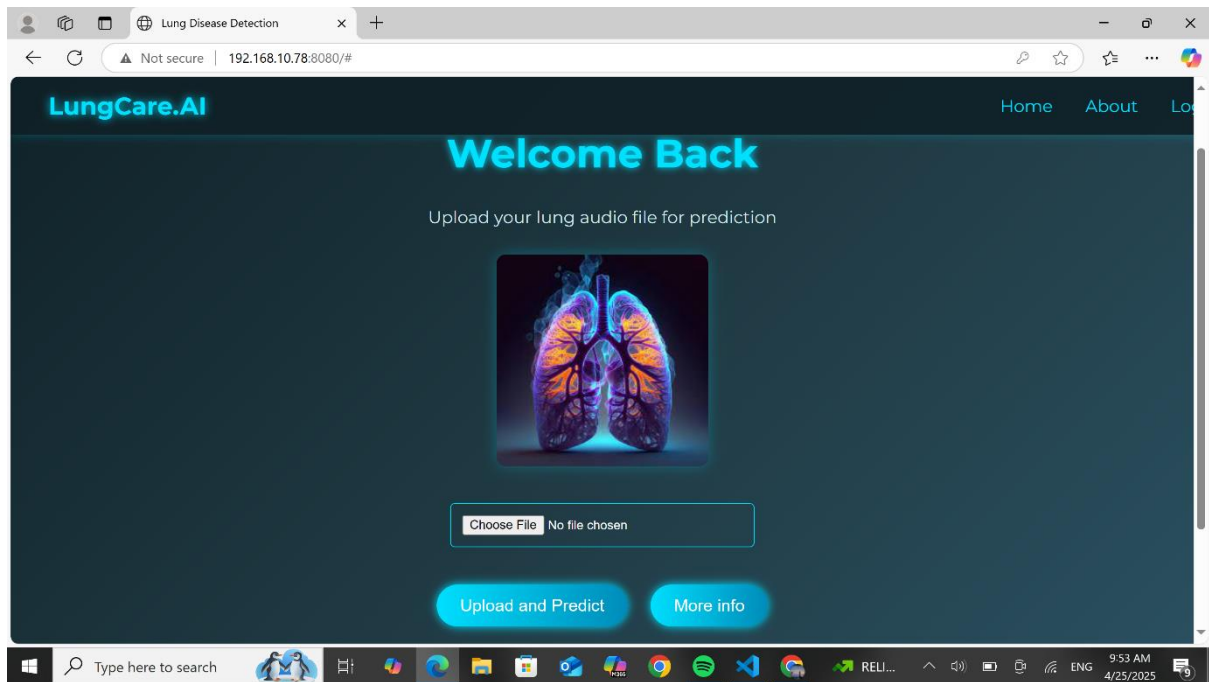


Figure 7.13: LungCare.AI Dashboard — Audio Upload for Prediction

Figure 7.13 is the lung sound upload interface of the LungCare.AI web app.

- Instruction: "Upload your lung audio file for prediction"
- File Upload: A button to choose and upload a lung sound (.wav/.mp3) file.
- Buttons:
 - Upload and Predict: Sends the audio to the backend for AI-based lung disease detection.
 - More Info: Likely provides additional details about the prediction process or supported file types.

This page allows users to submit lung audio recordings for AI-based respiratory disease diagnosis.

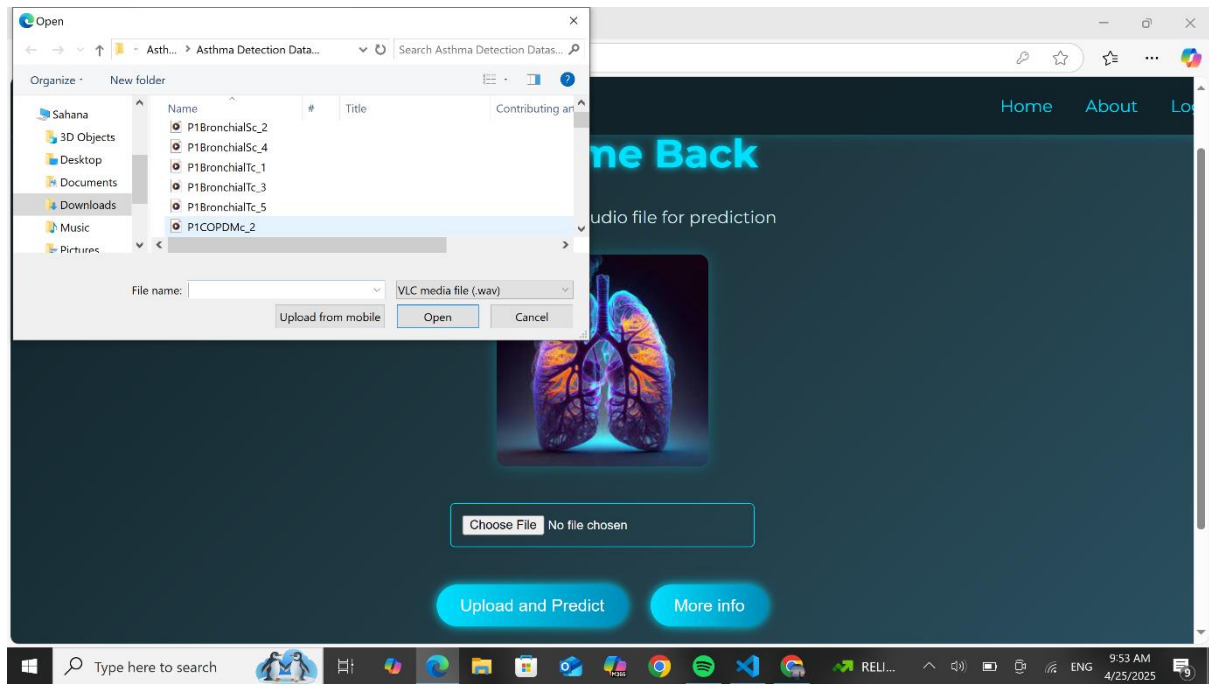


Figure 7.14: File Selection Dialog for Audio Upload

Figure 7.14 shows the file selection process for uploading a lung sound in the LungCare.AI application.

- Folder Opened: Asthma Detection Dataset
- File Format: .wav audio files (VLC media files) – commonly used for lung sound recordings.
- Selected File Example: P1COPDMc_2.wav, which likely corresponds to a COPD (Chronic Obstructive Pulmonary Disease) sample.
- Action Buttons:
 - Open: Uploads the selected file.
 - Cancel: Cancels the upload.

The user is in the process of selecting a lung audio file (likely for COPD or bronchial sound) from a local dataset to upload for AI prediction using the LungCare.AI system.

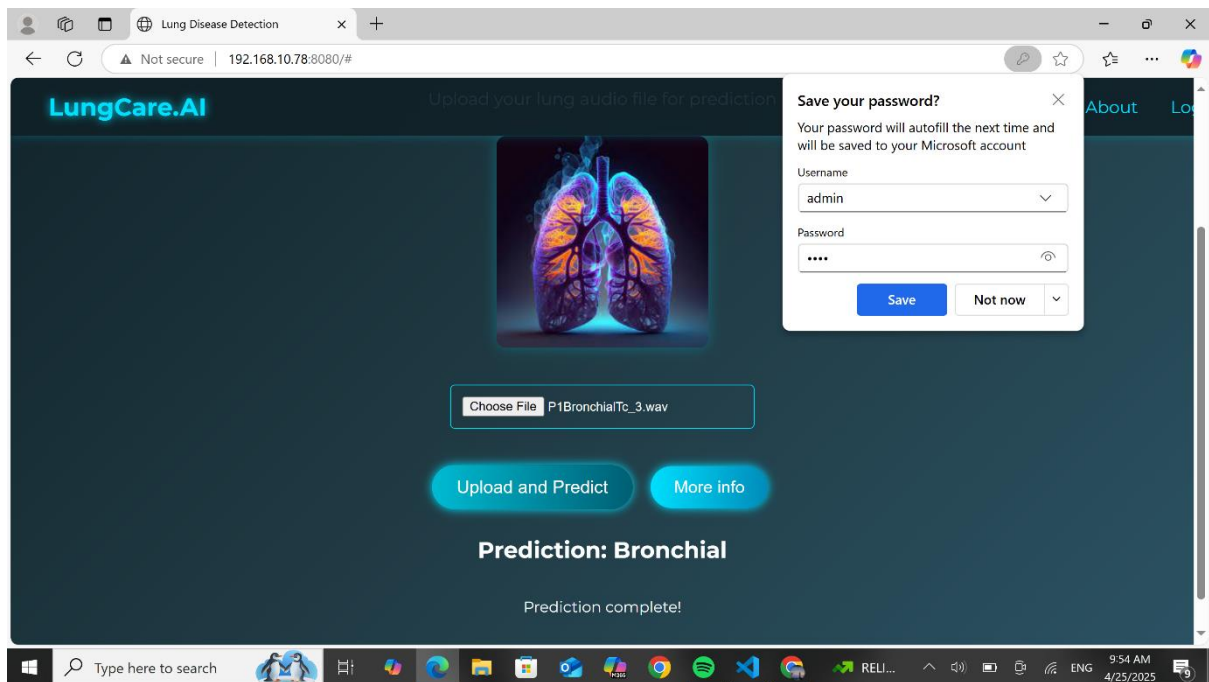


Figure 7.15: Prediction Result Display with Password-Prompt Overlay

Figure 7.15 shows a web application called LungCare.AI designed for lung disease detection from audio files of lung sounds.

- The user has selected an audio file named P1BronchialTc_3.wav (likely a lung sound recording).
- The user clicked the Upload and Predict button to analyze the lung audio.
- The system predicted "Bronchial" lung sounds, indicating a bronchial lung disease.
- A message says "Prediction complete!", confirming the analysis finished.
- There is also a prompt from the browser asking if the user wants to save the password for the username admin.

The web app allows uploading lung sound recordings (.wav files) and uses AI to detect the type of lung disease based on the audio. Here, it identified the presence of bronchial lung sounds.

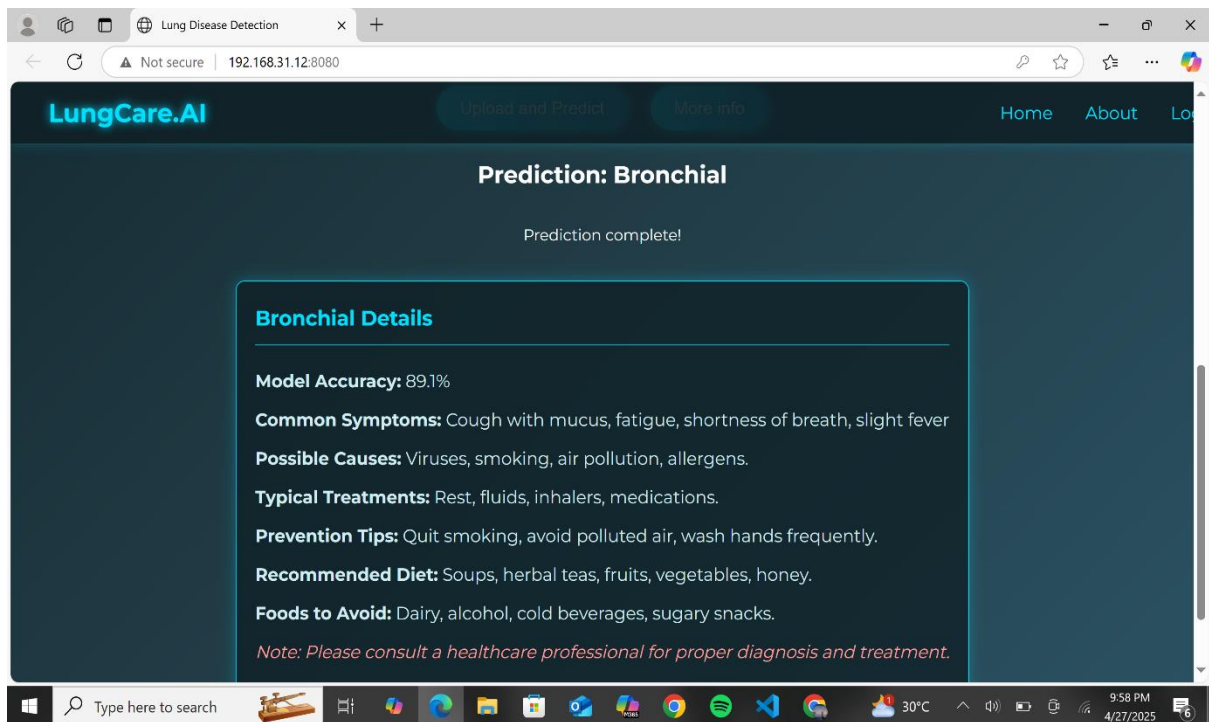


Figure 7.16: Detailed “More Info” Page for Prediction

Figure 7.16 shows a lung disease detection web application named LungCare.AI. The app has made a prediction that the user has Bronchial disease.

- Model Accuracy: 89.1%
- Common Symptoms: Cough with mucus, fatigue, shortness of breath, slight fever.
- Possible Causes: Viruses, smoking, air pollution, allergens.
- Typical Treatments: Rest, fluids, inhalers, medications.
- Prevention Tips: Quit smoking, avoid polluted air, wash hands frequently.
- Recommended Diet: Soups, herbal teas, fruits, vegetables, honey.
- Foods to Avoid: Dairy, alcohol, cold beverages, sugary snacks.

The app advises consulting a healthcare professional for proper diagnosis and treatment.

CONCLUSION

In this project, we investigated the feasibility of machine learning methods for respiratory disease classification from lung sound recordings. Recognizing the limitations of traditional diagnostic methods, especially in areas with limited access to experienced clinicians, we proposed an automated solution that can assist in the early and accurate diagnosis of respiratory conditions. The work entailed rigorous preprocessing of lung sound signals, wherein noise was alleviated and relevant features like Mel Frequency Cepstral Coefficients (MFCCs), zero-crossing rate, spectral centroid, and chroma features were mined to present the audio signals in a proper way. Based on these features, we trained an XGBoost classifier, a strong and efficient gradient boosting program that has a reputation for its speed, precision, and ability to support complex, noisy data. Our model has excellent classification accuracy and demonstrated excellent generalization ability via validation methods such as k-fold cross-validation. In addition to training the model, one key contribution of this work was establishing a web application where users can upload lung sound recordings and instantly obtain predictions of possible respiratory conditions. This real-world implementation not only verifies the technical viability of our model but also underlines the actual-world use value and practicability of the project for use in healthcare facilities. Web application guarantees availability of the instrument for healthcare personnel and patients so that screening of respiratory disease can be accelerated, facilitated, and made uniform. While promising results have been seen, we recognize the limitation of the study, i.e., that the requirement of large and diverse datasets for still better generalizability to unseen instances. Future improvements can include end-to-end learning, mobile app development for real-time diagnosis, and interaction with hospitals for real-world verification. In total, this project is a valuable contribution to the modernization of respiratory disease diagnosis using machine learning, audio processing, and digital technologies.

REFERENCE:

[1] Arka Roy, Udit Satija “RDLI Net: A Novel Lightweight Inception Network for Respiratory Disease Classification Using Lung Sounds”
IEEE Transactions on Instrumentation and Measurement VOLUME 72 ,2023
Journal Article, Publisher: IEEE.

[2] Wei-Bang Ma, Xiang-Yuan Deng, Yang Yang, Wai-Chi Fang “An Effective Lung Sound Classification System for Respiratory Disease Diagnosis Using densenet CNN Model with Sound Pre-processing Engine” IEEE Biomedical Circuits and Systems Conference (biocas), 2022 , Conference Paper,
Publisher: IEEE.

[3] Seyed Amir Latifi, Hassan Ghassemian, Maryam Imani “Feature Extraction and Classification of Respiratory Sound and Lung Diseases” 6th International Conference on Pattern Recognition and Image Analysis (IPRIA),2023
Conference Paper,Publisher: IEEE.

[4] Zizhao Chen, Hongliang Wang, Chia-Hui Yeh, Xilin Liu “Classify Respiratory Abnormality in Lung Sounds Using STFT and a Fine-Tuned ResNet18 Network” IEEE Biomedical Circuits and Systems Conference (BioCAS), 2022 ,Conference Paper ,Publisher: IEEE.

[5] Labiba Shayetreen, Afra Anani,Tasfia Mehnaz Tazin, Ummea Marzan, Syeda Raisa Afsar, Jannatun Noor, “A Comprehensive Respiratory Evaluation: Incorporating Lung Sound and Disease Classification Along with Spirometry Assessment” 6th International Conference on Electrical Engineering and

Information & Communication Technology (ICEEICT),2024,Conference Paper,
Publisher: IEEE.

[6] Nabila Husna Mohd Johari, Noreha Abdul Malik,Khairul Azami Sidek,
“Distinctive Features for Classification of Respiratory Sounds Between Normal
and Crackles Using Cepstral Coefficients" 7th International Conference on
Computer and Communication Engineering (ICCCE), 2018 ,Conference
Paper,Publisher: IEEE

DATASET COLLECTION:

<https://www.kaggle.com/datasets/mohammedtawfikmusaed/asthma-detection-dataset-version-2>

https://youtu.be/i-YF_043FqM?si=67zMavk_hBNGb5K7

APPENDIXES

PROGRAM:

BACKEND

Preprocessing:

```
import os
import librosa
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
import matplotlib
matplotlib.use('Agg')

# Path to dataset
dataset_path = "C:\\Users\\User\\Downloads\\XG booster\\Asthma
Detection Dataset Version 2-20250413T145858Z-001"

# Directory for spectrograms
spectrogram_dir = 'spectrogram_images'
os.makedirs(spectrogram_dir, exist_ok=True)

# Lists to store features and labels
features = []
labels = []

# Noise Reduction
```

```

def reduce_noise(y):
    noise_profile = np.median(y[:1000]) # More robust than mean
    return y - noise_profile

# Normalize Audio
def normalize_audio(y):
    return y / np.max(np.abs(y)) # More efficient than
librosa.util.normalize()

# Save Spectrogram with Scale
def save_spectrogram(segment, sr, filename):
    plt.figure(figsize=(8, 6))
    Sxx, _, _, _ = plt.specgram(segment, Fs=sr)
    plt.xlabel("Time (s)")
    plt.ylabel("Frequency (Hz)")
    plt.colorbar(label="Intensity (dB)") # Add scale
    plt.savefig(os.path.join(spectrogram_dir, filename),
bbox_inches='tight', pad_inches=0.1)
    plt.close()

# Feature Extraction
for file in os.listdir(dataset_path):
    if not file.endswith('.wav'): # Ignore non-audio files (e.g., .txt)
        continue

    file_path = os.path.join(dataset_path, file)

    # Extract label from filename

```

```
disease_label = file.split('_')[0] # Assuming label is the first part
before '_'
```

```
# Load audio
```

```
y, sr = librosa.load(file_path, sr=None, res_type='kaiser_fast')
```

```
# Preprocessing
```

```
y = reduce_noise(y)
```

```
y = normalize_audio(y)
```

```
# **Segment Length: 5 seconds (50% overlap) **
```

```
segment_length = 5 * sr # 5 seconds
```

```
hop_length = int (0.5 * segment_length)
```

```
for i in range (0, len(y) - segment_length + 1, hop_length):
```

```
    segment = y [i:i + segment_length]
```

```
    # MFCC Extraction
```

```
    mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=40,
hop_length=512)
```

```
    mfcc_mean = np.mean(mfcc, axis=1)
```

```
    # Save spectrogram with scale
```

```
    segment_filename = f"{file}_segment_{i}.png"
```

```
    save_spectrogram(segment, sr, segment_filename)
```

```
    # Store features
```

```
    features.append(mfcc_mean)
```

```
    labels.append(disease_label)
```

```

# Convert features to DataFrame
columns = [f'MFCC_{i}' for i in range (40)]
data = pd.DataFrame(features, columns=columns)
data['Disease'] = labels # Store disease names

# Standardization (Optimized)
scaler = StandardScaler()
data.iloc[:, :-1] = scaler.fit_transform(data.iloc[:, :-1])

# Save Preprocessed Data
data.to_csv('preprocessed_data.csv', index=False)
print ("Preprocessing completed successfully!")

```

Training:

```

import os
import librosa
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score,
recall_score, confusion_matrix, classification_report
from imblearn.over_sampling import SMOTE

```

```

import seaborn as sns
import joblib
import re

# Matplotlib headless backend
import matplotlib
matplotlib.use('Agg')

# ==== Paths ====
dataset_path = r"C:\Users\User\Downloads\XG booster\Asthma
Detection Dataset Version 2-20250413T145858Z-001\Asthma
Detection Dataset Version 2"
spectrogram_dir = 'spectrogram_images'
os.makedirs(spectrogram_dir, exist_ok=True)

# ==== Preprocessing Functions ====
def reduce_noise(y):
    noise_profile = np.median(y[:1000])
    return y - noise_profile

def normalize_audio(y):
    return y / np.max(np.abs(y))

def save_spectrogram(segment, sr, filename):
    plt.figure(figsize=(8, 6))
    plt.specgram(segment, Fs=sr)
    plt.xlabel("Time (s)")
    plt.ylabel("Frequency (Hz)")
    plt.colorbar(label="Intensity (dB)")

```



```

plt.savefig(os.path.join(spectrogram_dir, filename),
bbox_inches='tight', pad_inches=0.1)
plt.close()

def extract_label(file):
    # Use regex to find disease name in filename
    match =
re.search(r"(asthma|Bronchial|COPD|pneumonia|Healthy|wheezing
)", file, re.IGNORECASE)
    return match.group(0).capitalize() if match else "Unknown"

# ==== Feature Extraction ====
features = []
labels = []

for file in os.listdir(dataset_path):
    if not file.endswith('.wav'):
        continue

    file_path = os.path.join(dataset_path, file)
    disease_label = extract_label(file)

    if disease_label == "Unknown":
        print (f"[!] Skipping file with unknown label: {file}")
        continue

    y, sr = librosa.load(file_path, sr=None, res_type='kaiser_fast')
    y = reduce_noise(y)
    y = normalize_audio(y)

```

```

        segment_length = 5 * sr
        hop_length = int(0.5 * segment_length)

    for i in range(0, len(y) - segment_length + 1, hop_length):
        segment = y [i:i + segment_length]

        mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=40,
        hop_length=512)
        mfcc_mean = np.mean(mfcc, axis=1)

        features.append(mfcc_mean)
        labels.append(disease_label)

    # Optional: Save spectrogram
    segment_filename = f"{file}_segment_{i}.png"
    save_spectrogram(segment, sr, segment_filename)

# ==== Create DataFrame and Save ====
columns = [f"MFCC_{i}" for i in range(40)]
data = pd.DataFrame(features, columns=columns)
data['Disease'] = labels

# Save before training
data.to_csv('preprocessed_data.csv', index=False)

# ==== Training ====
# Handle missing values (if any)
data.dropna(inplace=True)

```

```

# Split features and labels
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

# Label Encoding
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, stratify=y_encoded, random_state=42
)

# Handle imbalance
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define models
xgb = XGBClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=7,
    subsample=0.8,

```

```

        colsample_bytree=0.8,
        random_state=42,
        use_label_encoder=False,
        eval_metric='mlogloss'
    )

rf = RandomForestClassifier(n_estimators=100, random_state=42)

ensemble_model = VotingClassifier(
    estimators=[('xgb', xgb), ('rf', rf)],
    voting='soft'
)

# Train
ensemble_model.fit(X_train, y_train)

# Predict
y_pred = ensemble_model.predict(X_test)

# Save model and transformers
joblib.dump(ensemble_model, 'respiratory_disease_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
joblib.dump(label_encoder, 'label_encoder.pkl')

# ==== Evaluation ====
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='weighted')
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')

```

```

conf_matrix = confusion_matrix(y_test, y_pred)
labels_list = label_encoder.classes_

# Confusion matrix plot
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
xticklabels=labels_list, yticklabels=labels_list)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# Classification report
class_report_dict = classification_report(y_test, y_pred,
target_names=labels_list, output_dict=True)
class_report_df = pd.DataFrame(class_report_dict).transpose()

plt.figure(figsize=(10, 5))
sns.heatmap(class_report_df.iloc[:-1, :-1], annot=True, cmap='coolwarm',
fmt='.2f')
plt.title('Classification Report')
plt.show()

# Print scores
print (f"✅ Accuracy: {accuracy:.2f}")
print (f"✅ F1 Score: {f1:.2f}")
print (f"✅ Precision: {precision:.2f}")
print (f"✅ Recall: {recall:.2f}")

```

```
print ("✅ Model, scaler, and encoder saved successfully!")
```

Implementation:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, f1_score, precision_score,
recall_score, confusion_matrix, classification_report
import joblib

# Load the dataset
data = pd.read_csv('preprocessed_data.csv')

# Extract features and labels
X = data.iloc[:, :-1].values # Features
y = data.iloc[:, -1].values # Disease names (before encoding)

# Load the trained label encoder
label_encoder = joblib.load('label_encoder.pkl')

# Convert labels to encoded format
y_encoded = label_encoder.transform(y)

# Split dataset into training & testing
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded,
test_size=0.2, random_state=42, stratify=y_encoded)
```

```

# Load the trained model
model = joblib.load('respiratory_disease_model.pkl')

# Predict on test data
y_pred = model.predict(X_test)

# Convert predictions back to disease names
predicted_diseases = label_encoder.inverse_transform(y_pred)
actual_diseases = label_encoder.inverse_transform(y_test)

# **Evaluation Metrics**
accuracy = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred, average='weighted') # Weighted F1 Score
precision = precision_score(y_test, y_pred, average='weighted') #
Weighted Precision
recall = recall_score(y_test, y_pred, average='weighted') # Weighted
Recall

# Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)

# Classification Report
class_report = classification_report(y_test, y_pred,
target_names=label_encoder.classes_)

# Print Evaluation Results
print(f"\nTest Accuracy: {accuracy:.2f}")
print(f"F1 Score: {f1:.2f}")

```

```

print (f"Precision: {precision:.2f}")
print (f"Recall: {recall:.2f}")
print ("\nConfusion Matrix:")
print(conf_matrix)
print ("\nClassification Report:")
print(class_report)

# Show first 5 predictions
print ("\nSample Predictions:")
for i in range (5):
    print (f"Predicted: {predicted_diseases[i]} | Actual:
    {actual_diseases[i]}")

```

Testing:

```

import numpy as np
import librosa
import joblib
from scipy.stats import mode

# Load model, scaler, and label encoder
model = joblib.
load('C:\\Users\\User\\Downloads\\prjt\\model\\respiratory_disease_mode
l.pkl')
scaler = joblib load('C:\\Users\\User\\Downloads\\prjt\\model\\scaler.pkl')
label_encoder =
joblib.load('C:\\Users\\User\\Downloads\\prjt\\model\\label_encoder.pkl')

# Test function with segmentation and majority voting

```



```

def predict_with_segments(audio_path):
    y, sr = librosa.load(audio_path, sr=None, res_type='kaiser_fast')

    # Preprocessing
    def reduce_noise(y): return y - np.median(y[:1000])
    def normalize_audio(y): return y / np.max(np.abs(y))

    y = reduce_noise(y)
    y = normalize_audio(y)

    segment_length = 5 * sr
    hop_length = int(segment_length * 0.5)

    predictions = []

    for i in range(0, len(y) - segment_length + 1, hop_length):
        segment = y[i:i + segment_length]
        mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=40,
hop_length=512)
        mfcc_mean = np.mean(mfcc, axis=1).reshape(1, -1)

        mfcc_scaled = scaler.transform(mfcc_mean)
        pred = model.predict(mfcc_scaled)[0]
        predictions.append(pred)

    if not predictions:
        return "Audio too short to process."

    final_pred = mode (predictions, keepdims=True) [0][0]

```

```

return label_encoder.inverse_transform([final_pred])[0]

# Test audio path

audio_path = "C:\\Users\\User\\Downloads\\prjt\\Asthma Detection
Dataset Version 2-20250413T145858Z-001\\Asthma Detection Dataset
Version 2\\P1COPDMc_2.wav"

# Prediction
result = predict_with_segments(audio_path)
print (f"✅ Final Predicted Disease (by voting): {result}")

```

FRONTEND:

HTML:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-
scale=1.0"/>
  <title>Lung Disease Detection</title>
  <link rel="stylesheet" href="style.css"/>
  <link
href="https://fonts.googleapis.com/css2?family=Montserrat:wght@400;7
00&display=swap" rel="stylesheet"/>
</head>

```

```

<body>

<!-- Navigation Bar -->
<nav class="navbar">
  <div class="nav-left">
    <span class="logo-text">LungCare.AI</span>
  </div>
  <div class="nav-right">
    <a href="#" class="nav-link" onclick="goHome()">Home</a>
    <a href="#" class="nav-link"
onclick="showAboutSection()">About</a>
    <a href="#" class="nav-link" onclick="logout()">Logout</a>
  </div>
</nav>

<!-- Main welcome section -->
<div class="container" id="mainSection">
  <h1 class="glow">LUNG DISEASE DETECTION</h1>
  <p class="subtitle">AI Powered Detection of Respiratory Disease Using
Lung Sounds</p>
  <div class="image-box">
    
  </div>
  <button onclick="showLoginForm()" class="login-
btn">Login</button>

</div>

<!-- Login section -->

```

```

<div class="container hidden" id="loginSection">
  <h2 class="glow">Welcome Back</h2>
  <p class="subtitle">Please login to continue</p>
  <input type="text" id="username" placeholder="Username"/><br>
  <input type="password" id="password" placeholder="Password"/><br>
  <button class="login-btn" onclick="login()">Submit</button>
  <p id="loginError" style="color: red;"></p>
</div>

<!-- Dashboard section -->
<div class="container hidden" id="dashboardSection">
  <h2 class="glow">Welcome Back</h2>
  <p class="subtitle">Upload your lung audio file for prediction</p>
  <div class="image-box">
    
  </div>

  <!-- Form and buttons -->
  <form id="uploadForm" enctype="multipart/form-data">
    <input type="file" id="audioFile" name="audio" accept=".wav"
required /><br><br>
    <div class="button-row">
      <button type="button" class="login-btn"
onclick="predictDisease()">Upload and Predict</button>
      <button type="button" class="login-btn"
onclick="showMoreInfo()">More info</button>
    </div>
  </form>

  <h2>Prediction: <span id="predictionResult">...</span></h2>

```

<p id="result"></p>

<!-- Extra info box -->

<div id="moreInfo" class="subtitle" style="margin-top: 20px; display: none;"></div>

<p id="diseaseDetailsText"></p>

</div>

<!-- About section -->

<div id="aboutSection" class="container hidden">

<h2 class="glow">About LungCare.AI</h2>

<p class="subtitle">

LungCare.AI is an intelligent web application designed to assist in the early detection of respiratory diseases by analyzing lung sound patterns. Using advanced audio signal processing and powerful machine learning algorithms like XGBoost and RandomForest, the system extracts MFCC features from lung audio and predicts the most probable condition.

</p>

<p class="subtitle">

The application leverages AI to identify diseases such as asthma, bronchitis, pneumonia, and other lung-related conditions with high accuracy. Our aim is to provide an accessible and fast screening tool for doctors, healthcare workers, and individuals, especially in remote or underserved regions.

</p>

<p class="subtitle">

Simply upload a lung sound recording (in .wav format), and let our AI model analyze and return a prediction in seconds. LungCare.AI combines

innovative healthcare technology with a user-friendly interface, all while maintaining data privacy and efficiency.

</p>

</div>

<script src="script.js"></script>

</body>

</html>

STYLE CSS:

/* Base Styles */

body {

margin: 0;

padding: 0;

font-family: 'Montserrat', 'Segoe UI', sans-serif;

background: linear-gradient(135deg, #0f2027, #203a43, #2c5364);

color: white;

min-height: 100vh;

}

/* Navbar */

.navbar {

position: fixed;

top: 0;

width: 100%;

z-index: 1000;

display: flex;

justify-content: space-between;

```
align-items: center;
padding: 15px 40px;
background: rgba(15, 32, 39, 0.9);
font-size: 18px;
box-shadow: 0 2px 10px rgba (0, 224, 255, 0.2);
}
```

```
.nav-left. logo-text {
  font-weight: bold;
  font-size: 26px;
  color: #00e0ff;
  text-shadow: 0 0 10px #00e0ff;
}
```

```
.nav-right a {
  color: #00e0ff;
  margin-left: 30px;
  text-decoration: none;
  transition: color 0.3s ease;
}
```

```
.nav-right a: hover {
  color: #00bcd4;
}
```

```
/* Container */
.container {
  display: flex;
  justify-content: center;
```

```
align-items: center;
flex-direction: column;
min-height: 80vh;
padding: 100px 20px 20px;
text-align: center;
box-sizing: border-box;
}
```

```
/* Heading */
h1, h2 {
  font-size: 42px;
  color: #00e0ff;
  text-shadow: 0 0 8px #00e0ff;
  margin-bottom: 10px;
}
```

```
/* Subheading */
.subtitle {
  font-size: 18px;
  margin-bottom: 25px;
  color: #d1ecf1;
  max-width: 800px;
  line-height: 1.6;
}
```

```
/* Image */
.image-box img {
  max-width: 300px;
```



```

width: 100%;
border-radius: 10px;
margin-bottom: 25px;
box-shadow: 0 0 15px rgba(0, 224, 255, 0.3);
}

/* Login Button */
.login-btn {
padding: 12px 25px;
font-size: 18px;
color: white;
background: linear-gradient(to right, #00e0ff, #008cba);
border: none;
border-radius: 30px;
cursor: pointer;
box-shadow: 0 0 10px #00e0ff;
transition: all 0.3s ease;
margin: 10px;
}

.login-btn:hover {
background: linear-gradient(to right, #00bcd4, #005f73);
transform: scale(1.05);
}

/* Inputs */
input {
padding: 12px;
margin: 10px;

```

```

width: 80%;
max-width: 300px;
border-radius: 5px;
border: 1px solid #00e0ff;
background: rgba(255);
}

.hidden {
display: none;
}

.toast {
position: fixed;
bottom: 30px;
left: 50%;
transform: translateX(-50%);
background-color: #111;
color: #fff;
padding: 12px 24px;
border-radius: 8px;
font-size: 14px;
z-index: 9999;
opacity: 0;
pointer-events: none;
transition: opacity 0.4s ease;
}

.toast.show {
opacity: 1;
pointer-events: auto;
}

```

```
}
```

```
/* Add this to your style.css */
```

```
#moreInfo {  
    background: rgba(15, 32, 39, 0.8);  
    border-radius: 10px;  
    padding: 20px;  
    margin-top: 20px;  
    max-width: 800px;  
    text-align: left;  
    border: 1px solid #00e0ff;  
    box-shadow: 0 0 15px rgba(0, 224, 255, 0.3);  
}
```

```
#moreInfo h3 {  
    color: #00e0ff;  
    margin-top: 0;  
    border-bottom: 1px solid #00e0ff;  
    padding-bottom: 10px;  
}
```

```
#moreInfo p {  
    margin: 10px 0;  
    line-height: 1.6;  
}
```

```
.note {  
    color: #ff9e9e;  
    font-style: italic;  
}
```

JAVA SCRIPT:

```
document.addEventListener('DOMContentLoaded', function () {  
    // Set home as default view  
    goHome ();  
    // Nav link events  
    document. querySelector('.nav-link.home')?.addEventListener('click',  
goHome);  
    document. querySelector('.nav-link.about')?.addEventListener('click',  
showAboutSection);  
    document. querySelector('.nav-link.logout')?.addEventListener('click',  
logout);  
  
    // Button events  
    document.getElementById('predictBtn')?.addEventListener('click',  
predictDisease);  
    document. getElementById('moreInfoBtn')?.addEventListener('click',  
showMoreInfo);  
});  
  
function showSection(sectionId) {  
    const sections = document.querySelectorAll('.container');  
    sections.forEach(sec => sec.classList.add('hidden'));  
    document.getElementById(sectionId)?.classList.remove('hidden');  
}
```

APP.PY

```
from flask import Flask, request, jsonify
from werkzeug.utils import secure_filename
import joblib
import librosa
import numpy as np
import os
from flask_cors import CORS
from scipy.stats import mode

app = Flask(__name__)
CORS(app)

# === CONFIGURATION ===
UPLOAD_FOLDER = 'temp'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

# === LOAD MODEL AND TRANSFORMERS ===
model = joblib.load('C:\\Users\\User\\Documents\\new
one\\backend\\model\\respiratory_disease_model.pkl')
scaler = joblib.load('C:\\Users\\User\\Documents\\new
one\\backend\\model\\scaler.pkl')
label_encoder = joblib.load('C:\\Users\\User\\Documents\\new
one\\backend\\model\\label_encoder.pkl')

print("[INFO] Model, scaler, and label encoder loaded successfully.")
```

```

# === AUDIO PROCESSING FUNCTIONS ===

def reduce_noise(y):
    return y - np.median(y[:1000])

def normalize_audio(y):
    return y / np.max(np.abs(y))

def extract_features_with_segments(audio_path, segment_duration=5):
    y, sr = librosa.load(audio_path, sr=None, res_type='kaiser_fast')
    y = reduce_noise(y)
    y = normalize_audio(y)

    segment_len = segment_duration * sr
    hop_len = int(0.5 * segment_len)

    predictions = []

    for i in range(0, len(y) - segment_len + 1, hop_len):
        segment = y[i:i + segment_len]
        mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=40,
hop_length=512)
        mfcc_mean = np.mean(mfcc, axis=1).reshape(1, -1)

        mfcc_scaled = scaler.transform(mfcc_mean)
        pred = model.predict(mfcc_scaled)[0]
        predictions.append(pred)

    if not predictions:
        return None # Audio too short

```

```

final_prediction = mode (predictions, keepdims=True) [0][0]
    return label_encoder.inverse_transform([final_prediction])[0]

# === ROUTES ===
@app. route('/')
def home ():
    return "Lung Disease Detection API is running!"
@app. route ('/predict', methods=['POST'])
def predict ():
    try:
        print ("Received a file for prediction.")
        audio_file = request.files['file']
        filename = secure_filename(audio_file.filename)
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        audio_file.save(filepath)

        print ("Extracting features...")
        prediction = extract_features_with_segments(filepath)
        print (f"Prediction made: {prediction}")

        os.remove(filepath) # Clean up after prediction
        return jsonify({'prediction': prediction})
    except Exception as e:
        print (f"Error: {str(e)}")
        return jsonify({'error': str(e)})
# === MAIN ===
if __name__ == '__main__':
    app.run(debug=True)

```